Contents

Notes on Cosmological Simulations

This is a living document—a collection of knowledge that I have gathered while learning about cosmological simulations. It is not a formal text but rather a journal, an attempt to solidify concepts by structuring and explaining them in my own way.

Along the way, I have been developing a proof-of-concept N-body/hydrodynamics simulation, which allowed me to understand algorithms by implementing them, and to appreciate physical principles by seeing their effects in a virtual universe. The explanations in this document are reflected in this practical work.

This is my best effort to present this knowledge in the way that I would have found most helpful at the start of my learning process.

Victor Alamo
vialamo@gmail.com
https://github.com/vialamo/nbody

Introduction

At its core, a cosmological simulation is a computational time machine. Because astrophysicists cannot physically experiment on stars or galaxies in a laboratory, they use computers to build virtual patches of the cosmos. By seeding a simulation volume with the initial mathematical conditions of the Big Bang and stepping it forward in time using the laws of physics, we can watch 13.8 billion years of cosmic evolution unfold in a matter of days or weeks.

The Origins

The endeavor to simulate the cosmos began in the 1970s. Early pioneers, such as Jim Peebles, ran the very first N-body simulations using just a few hundred particles to study how galaxies might cluster together under gravity. By the 1980s, researchers were running the first 3-dimensional models of Cold Dark Matter (CDM). These early simulations were severely limited by the computers of their time; they treated entire galaxies as single points of mass and modeled only the force of gravity, completely ignoring the complex fluid dynamics of gas.

Modern Applications

Today, cosmological simulations are some of the most intensive computational tasks on Earth. Projects like the Millennium Simulation, Illustris, and EAGLE utilize supercomputing clusters to track billions—and sometimes trillions—of particles.

Nowadays, these virtual universes are used as the primary theoretical laboratories for astrophysics, serving three main purposes:

- **Testing the Standard Model:** We can easily alter the parameters of a simulation—adding more dark energy, changing the mass of dark matter particles, or altering the laws of gravity. By comparing the resulting "mock universe" to the real one we see through telescopes, we can prove or disprove fundamental theories of physics.
- **Calibrating Telescope Data:** Massive modern observatories (like the James Webb Space Telescope and the Euclid satellite) gather an overwhelming amount of data. Simulations provide the theoretical maps required to interpret those observations, helping astronomers distinguish between optical illusions (like redshift-space distortions) and true physical structures.
- **Probing the Unobservable:** While a telescope can only capture a frozen snapshot of a galaxy at one specific moment in its life, a simulation allows us to continuously watch the dynamic, billion-year processes of galaxy mergers, black hole feeding, and cosmic web formation from beginning to end.

The Anatomy of a Cosmological Simulation

Before diving into the specific mathematics and algorithms, it is helpful to outline exactly what we are trying to simulate. A modern cosmological simulation is not a single, monolithic algorithm. Rather, it is a carefully coupled system of different physical frameworks designed to mimic the actual composition and behavior of the universe.

To build a realistic virtual patch of the cosmos, a simulation must continuously balance three distinct computational pillars:

- **Collisionless Dark Matter :** Dark matter accounts for roughly 85% of the matter in the universe and dominates its gravitational landscape. Because it does not interact with light or experience thermodynamic pressure, we treat it as a "collisionless" fluid. Computationally, this is handled by an **N-body solver**, which tracks millions (or billions) of discrete, massive particles moving purely under the influence of gravity to form the filaments and halos of the cosmic web.
- **Baryonic Gas:** Normal, visible matter (hydrogen and helium) behaves very differently from dark matter. Gas clouds crash into each other, heat up, form shockwaves, and exert pressure. To simulate this, gravity alone is insufficient; we require a **Hydrodynamics solver**. This typically involves dividing the simulation volume into a 3D grid (an Eulerian approach) to strictly track the conservation of mass, momentum, and energy as the fluid flows through space.
- **The Expanding Background:** Unlike a simulation of a single, static solar system, a cosmological simulation takes place in an expanding universe. The coordinate grid itself stretches over time. Both the N-body particles and the hydrodynamic gas must be subjected to a cosmological background model (driven by the Friedmann equations) to properly account for the dilution of density and the slowing of velocities due to cosmic expansion.

Because gravity is the universal language that links the dark matter and the gas together, it is the most critical component to get right. We will begin by breaking down the foundational engine that drives the formation of all large-scale structure: calculating the gravitational pull of N interacting particles.

The Major Cosmological Codes

The global astrophysical community relies on a handful of massive, highly optimized, open-source software packages to run these supercomputer simulations.

It is helpful to know that almost all modern codes are split by their philosophical approach to fluid dynamics: some treat gas as a collection of individual moving particles (**Lagrangian** methods like SPH), while others treat gas as a fluid flowing through a fixed or adaptive 3D grid (**Eulerian** methods like AMR).

Here is a summary of the most prominent cosmological codes used in modern research and the underlying engines that drive them:

Code Name	Gravity Solver	Hydrodynamics Solver	Notable Simulations
GADGET	TreePM (Tree-based + Particle-Mesh)	SPH (Smoothed Particle Hydrodynamics)	Millennium, EAGLE
RAMSES	AMR-coupled Particle-Mesh	AMR (Adaptive Mesh Refinement)	Horizon-AGN
ENZO	Particle-Mesh	AMR (Adaptive Mesh Refinement)	Renaissance Simulations
AREPO	TreePM	Moving Voronoi Mesh (Unstructured grid)	Illustris, IllustrisTNG
SWIFT	Fast Multipole Method (Tree)	Modern SPH / Meshless Finite Mass	Flamingo

These codes represent decades of collaborative optimization. We will dive into the core mechanics of how gravity and gas interact on a discrete grid in the following chapters.

The N-Body Problem

The **N-body problem** is the task of predicting the dynamical evolution of a system composed of N particles that interact through mutual gravitational attraction. Each particle experiences the combined gravitational

influence of all others, and because these forces depend on the instantaneous positions of every particle, the motion of any one particle cannot be determined independently.

In Newtonian gravity, the equation of motion for particle i with position vector $\mathbf{x}_i$, velocity $\mathbf{v}_i$, and mass m_i is:

$$m_i \frac{d^2\mathbf{x}_i}{dt^2} = -Gm_i \sum_{\substack{j=1 \\ j\neq i}}^{N} m_j \frac{\mathbf{x}_i - \mathbf{x}_j}{|\mathbf{x}_i - \mathbf{x}_j|^3}$$

This coupled system of $3N$ second-order differential equations has no general analytic solution for $N > 2$, making it one of the foundational challenges of computational astrophysics.

Direct Summation

The most straightforward numerical method for solving the N-body problem is the **direct-summation algorithm**, which explicitly computes the gravitational force on each particle from every other particle. The procedure for a single time step can be described as follows:

1. **Select a particle**, say particle A.
2. **Loop over all other particles** $(B, C, D, \ldots)$.
3. **Compute the pairwise force** on A from each other particle using Newton's law of gravitation:

$$\mathbf{F}_{AB} = -G\frac{m_A m_B}{r_{AB}^2}\hat{\mathbf{r}}_{AB}$$

   where $\mathbf{r}_{AB} = \mathbf{x}_A - \mathbf{x}_B$ and $\hat{\mathbf{r}}_{AB} = \mathbf{r}_{AB}/|\mathbf{r}_{AB}|$.
4. **Sum all pairwise forces** to obtain the total force on particle A:

$$\mathbf{F}_A = \sum_{\substack{B=1 \\ B\neq A}}^{N} \mathbf{F}_{AB}$$

5. **Update** particle A's position and velocity using this total force (via an integration method such as Velocity Verlet).
6. **Repeat** the process for every particle in the system.

Although conceptually simple and physically exact, the direct-summation method is computationally prohibitive for large N. To compute the total force on one particle, we must evaluate $N - 1$ pairwise interactions. Doing this for all N particles requires approximately $N(N-1) \approx N^2$ force evaluations per time step.

In computational complexity terms, this corresponds to $O(N^2)$ scaling — meaning that doubling the number of particles multiplies the total computational cost by roughly four. This quadratic growth rapidly becomes intractable: while $N \sim 10^3$ is easily manageable, $N \sim 10^6$ would require on the order of 10^{12} pairwise force evaluations per step.

Because of this steep scaling, the direct method is impractical for cosmological simulations, which often involve millions or billions of particles. To overcome this, we rely on **approximation schemes** — such as the **Particle-Mesh (PM)** and **Particle-Particle Particle-Mesh (P³M)** — that preserve physical accuracy while reducing computational cost from $O(N^2)$ to nearly $O(N \log N)$ or better.

Boundaries and Singularities

Two fundamental problems arise when trying to model gravity in a computer. The first is how to simulate an infinite universe in a finite box, and the second is how to handle the infinite force that occurs when two particles get too close.

Periodic Boundary Conditions

To simulate a small, representative patch of an infinite, uniform universe without having particles react to artificial "walls", simulations employ **periodic boundary conditions**. This method treats the simulation space as a seamless, repeating tile.

A particle exiting one face immediately re-enters from the opposite face. This means that when calculating the force between two particles, the "wrap-around" distance must be considered. We always use the shortest

path between the two particles. This is known as the **Minimum Image Convention**, and it ensures that no particle ever feels an artificial "edge of the universe."

Gravitational Softening

Newton's law of gravity, $F \propto 1/r^2$, has a mathematical singularity: as the distance r between two particles approaches zero, the force between them approaches infinity. In a simulation that moves in discrete time steps, these immense forces can cause particles to be catapulted away at unrealistic speeds, completely wrecking the simulation's stability and energy conservation.

To prevent this, we introduce **gravitational softening**. This technique modifies Newton's law of gravity by adding a parameter known as the **softening length**, ϵ (epsilon), to the denominator:

$$F = \frac{Gm_1m_2}{r^2 + \epsilon^2}$$

This simple addition ensures the denominator can never be zero. When particles are far apart, they feel the normal $1/r^2$ force. However, when their separation becomes comparable to or smaller than ϵ, the force is "softened" and stops growing, leveling off at a large but finite value.

A common and effective rule of thumb is to base the softening length on the **mean inter-particle spacing**, d. For a box with side L and N particles, it's calculated as:

$$d = \frac{L}{N^{1/3}}$$

A typical choice for the softening length is then a small fraction of this, such as $\epsilon = d/30$. This ensures that the force is physically accurate for the vast majority of interactions, while the softening only activates during rare, close encounters to prevent numerical catastrophe.

Key Literature & Further Reading
Springel, V. (2005). *The cosmological simulation code GADGET-2. Monthly Notices of the Royal Astronomical Society*, 364(4), 1105-1134. arXiv:astro-ph/0505010. Available at https://arxiv.org/abs/astro-ph/0505010

The Integrator

The Euler Method

To move the particles through time, we need an "integrator"—an algorithm that takes the current state of a particle (its position and velocity) and predicts its state a small moment later. The most intuitive and straightforward approach is the **Euler method**.

The Euler method assumes that the velocity and acceleration are constant over one small time step, Δt. It calculates the force on the particle at its current position to find its acceleration, and then takes a linear step forward.

The update equations are:

1. **Update Position:** $\mathbf{x}_{n+1} = \mathbf{x}_n + \mathbf{v}_n\Delta t$
2. **Update Velocity:** $\mathbf{v}_{n+1} = \mathbf{v}_n + \mathbf{a}_n\Delta t$

While simple, the Euler method's core assumption is almost always wrong. In a gravitational system, the force is constantly changing as a particle moves, but the Euler method is blind to any changes that occur during the step.

This error, while tiny on each step, is **systematic**. It always pushes the energy in the same direction. Over thousands of steps, this causes a simulated planet to slowly spiral outwards, gaining energy with every orbit until it eventually flies away. This failure to conserve energy makes the Euler method unsuitable for any simulation where long-term stability is important.

Velocity Verlet

The failure of the Euler method shows that a more robust integrator is needed—one that accounts for the fact that forces change *during* a time step. An effective solution is an algorithm called **Velocity Verlet**.

The core idea is to use a more accurate, averaged acceleration to update the velocity. Instead of just using the acceleration from the beginning of the step, it uses the average of the accelerations from the beginning and the end of the step.

The algorithm proceeds in three steps:

1. **Calculate the New Position:** First, advance the position using the current velocity and acceleration.
$$\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + \mathbf{v}(t)\Delta t + \frac{1}{2}\mathbf{a}(t)\Delta t^2$$
2. **Calculate the New Acceleration:** With the new position, calculate the new force vector $\mathbf{F}(\mathbf{x}(t + \Delta t))$ and from it, the new acceleration.
$$\mathbf{a}(t + \Delta t) = \frac{\mathbf{F}(\mathbf{x}(t + \Delta t))}{m}$$
3. **Calculate the New Velocity:** Finally, update the velocity using the **average** of the old acceleration $\mathbf{a}(t)$ and the new acceleration $\mathbf{a}(t + \Delta t)$.
$$\mathbf{v}(t + \Delta t) = \mathbf{v}(t) + \frac{\mathbf{a}(t) + \mathbf{a}(t + \Delta t)}{2}\Delta t$$

This final step of averaging the accelerations is the key. It corrects for the systematic drift of the Euler method, and it is what makes Velocity Verlet a **symplectic integrator**. This crucial property is what enables the algorithm to produce a stable and accurate trajectory, conserving energy remarkably well over long periods.

Symplectic Integration

A **symplectic integrator** is an algorithm specifically designed to respect the underlying geometry of physics, a property that allows it to conserve a system's total energy over very long periods. The practical importance of this is best understood by comparing how different integrators handle a simple gravitational problem, like a planet orbiting a star.

- A **non-symplectic** integrator, like the Euler method, consistently makes an error in the same direction. It always "cuts the corner" of the orbit, pushing the planet slightly outwards. These errors add up, causing the planet's energy to systematically increase and its orbit to spiral away.
- A **symplectic** integrator, like Verlet, makes errors that are correlated. On one step, it might slightly overshoot the true orbit, but on a later step, it will undershoot it to compensate. The errors effectively cancel each other out over time.

Instead of a catastrophic spiral, the simulated planet executes a stable "wobble" along the correct orbital path. The shape of the orbit might oscillate slightly, but its average size and, crucially, its average energy, remain correct for millions of steps.

The deeper reason for this remarkable stability lies in a concept from classical mechanics called **phase space**. Phase space is an abstract map where every point represents the complete state of a particle—both its **position** and its **momentum**. For a system where energy is conserved, a fundamental rule known as **Liouville's Theorem** states that the "area" (or volume) of any group of states in phase space must stay constant as the system evolves.

Symplectic integrators are mathematically constructed to **perfectly preserve this phase space volume**. Because they respect this fundamental geometric rule, they are forbidden from having the systematic energy drift that plagues simpler methods. The bounded energy error (the "wobble") is a direct consequence of this property.

This is why symplectic integrators are chosen over the Euler method for any long-term simulation of a conservative system.

The Kick-Drift-Kick Integrator

The standard Velocity Verlet integrator is a powerful tool, but it was designed for a universe with static rules. The introduction of cosmic expansion adds a velocity-dependent term to the equations of motion (the "Hubble drag", explored in a later section). This new term creates a challenge for the standard Verlet algorithm because its symplectic nature is strictly defined for forces that depend only on position, not velocity. To handle this

new term gracefully, we adopt a different formulation known as a **Leapfrog** scheme. The most common implementation, the **Kick-Drift-Kick (KDK)** integrator, is the standard choice in cosmological simulations.

The KDK scheme advances the system from a time t to $t + \Delta t$ in three stages:

1. First "Kick" (Velocity Half-Step)

The velocities are "kicked" forward by half a time step using the acceleration from the beginning of the step.

$$\mathbf{v}(t + \tfrac{1}{2}\Delta t) = \mathbf{v}(t) + \mathbf{a}(t)\frac{\Delta t}{2}$$

2. "Drift" (Position Full-Step)

The positions then "drift" for a full time step using the more accurate **mid-step velocity**.

$$\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + \mathbf{v}(t + \tfrac{1}{2}\Delta t)\Delta t$$

3. Second "Kick" (Velocity Second Half-Step)

Finally, the new acceleration, $\mathbf{a}(t + \Delta t)$, is computed from the forces at the new positions, $\mathbf{x}(t + \Delta t)$, and the **mid-step velocity**, $\mathbf{v}(t + \frac{1}{2}\Delta t)$. The acceleration is then used to complete the velocity update for the second half of the time step.

$$\mathbf{v}(t + \Delta t) = \mathbf{v}(t + \tfrac{1}{2}\Delta t) + \mathbf{a}(t + \Delta t)\frac{\Delta t}{2}$$

While mathematically equivalent to Verlet in simpler cases, this staggered formulation is particularly robust for handling the time-varying and velocity-dependent forces present in a cosmological simulation. The symmetric "kick-force-kick" structure gracefully incorporates these complexities, which is why the KDK leapfrog is the workhorse integrator for nearly all modern cosmological N-body codes.

Key Literature & Further Reading
Tsang, D., Galley, C. R., Stein, L. C., & Turner, A. (2015). *Slimplectic Integrators: Variational Integrators for General Nonconservative Systems.* arXiv:1506.08443. Available at https://arxiv.org/pdf/1506.08443.pdf.

The Particle-Mesh Method

The Particle-Mesh (PM) method is built on a different perspective. Instead of calculating the gravitational pull between every pair of particles, it simplifies the problem by describing the mass distribution on a regular grid. From this "mass map", the gravitational potential and forces can be solved on the grid itself. These are the steps:

1. **Potential calculation:** First, the gravitational potential (Φ) is calculated for the entire grid. The potential is a scalar "landscape" that describes the depth of the gravitational well at every point.
2. **Force calculation:** Second, the force ($\mathbf{F}$) is determined by finding the steepest downhill slope (the gradient) of that potential landscape.

This `Mass -> Potential -> Force` pipeline is the foundation of the PM method. The following sections break down how each part of this process is achieved.

Step 1. Finding the Potential

The process of finding the potential begins by describing the mass distribution on the grid, which then serves as the input for the physical law that governs how that mass creates the potential.

Mass Assignment (NGP)

The first step in this process is **mass assignment**: the procedure for transferring the mass of our continuously positioned particles onto the discrete nodes of the grid.

The simplest and most intuitive way to do this is the **Nearest Grid Point (NGP)** scheme: for each particle, we find the single grid point (or cell center) that it is closest to, and assign the particle's *entire mass* to that one point.

The result is an array representing the mass density field, $\rho_{i,j,k}$. Mathematically, the density in a given cell (i, j, k) is the sum of the masses of all particles within that cell, divided by the cell's volume:

$$\rho_{i,j,k} = \frac{1}{L^3} \sum_{p \in \text{cell}(i,j,k)} m_p$$

Where m_p is the mass of a particle p, and L is the side length of a grid cell.

While NGP is very simple, it can introduce inaccuracies. As we will explore in a later section, more sophisticated schemes like Cloud-in-Cell (CIC) can be used to create a smoother and more accurate density field.

Poisson's Equation

The potential field Φ can be determined from the mass density field, $\rho_{i,j,k}$. The fundamental law linking mass to gravitational potential is **Poisson's Equation**.

$$\nabla^2 \Phi = 4\pi G \rho$$

Where:

- ρ is the mass density grid — the **input**.
- Φ is the gravitational potential field — the **output**.
- G is the gravitational constant.
- ∇^2 (the **Laplacian**) is a mathematical operator that measures how much a function curves around a point—the **net curvature**.

In this equation, mass acts as the source of curvature: where there is mass, the potential bends inward, forming gravitational wells that attract other masses. Where $\rho = 0$, there's no net curvature. This may seem counterintuitive, as the potential field clearly forms a curved, gravitational well even in the empty space around a mass. The key is that the Laplacian, $\nabla^2\Phi$, measures the **net curvature**. In the smooth $1/r$ shape of a potential in empty space, the radial inward bending of the field is perfectly balanced by a natural geometric spreading effect in three dimensions. These two effects cancel each other out, resulting in zero net curvature.

Solving Poisson's equation means finding the global shape of Φ given all the local sources ρ. Doing this directly is computationally expensive, but as we'll see next, the Fast Fourier Transform (FFT) offers an efficient way to compute it by turning this differential problem into simple multiplications in frequency space.

The FFT and the Convolution Theorem

Given the mass density grid, ρ, and the rule connecting it to the potential, Poisson's Equation, the challenge now is to solve it. Calculating the potential at every grid point by summing the influence from all other grid points is a "brute-force" operation known as a **convolution**. It's a slow, computationally expensive task.

Fortunately, there is a more efficient mathematical tool that can solve this problem: the **Fast Fourier Transform (FFT)**. This algorithm is used to take advantage of the **Convolution Theorem**.

The Fourier Transform The **Fourier Transform** is a mathematical operation that rewrites any spatial field in terms of its **spatial frequencies** — the underlying wave patterns that make it up.

Suppose we have a density field $\rho(\mathbf{x})$ defined across space. Instead of describing it point by point, we can express it as a sum of smooth oscillating patterns — *plane waves* — each with its own wavelength, direction, and amplitude. The Fourier Transform tells us **how each wave** contributes to the total field.

Formally, it is written as:

$$\hat{\rho}(\mathbf{k}) = \int \rho(\mathbf{x}) e^{-i\mathbf{k}\cdot\mathbf{x}} d^3x$$

Here, $\mathbf{x}$ represents position in real space, and $\mathbf{k}$ is the **wavevector**, describing one of those plane waves. Each component (k_x, k_y, k_z) measures how rapidly the field oscillates along that direction, while its magnitude

$$k = |\mathbf{k}| = \frac{2\pi}{\lambda}$$

tells us the *spatial frequency*.

The result of the transform, $\hat{\rho}(\mathbf{k})$, is a **complex number**. Its magnitude $|\hat{\rho}(\mathbf{k})|$ gives the *amplitude*, and its phase $\arg(\hat{\rho}(\mathbf{k}))$ specifies the *offset* — where the wave's peaks and troughs occur in space.

Thus, while $\rho(\mathbf{x})$ is a **real-valued function of position**, $\hat{\rho}(\mathbf{k})$ is a **complex-valued function of wavevector**. They describe the same field, but from two complementary perspectives: one in **Real space**, one in **Frequency (or k-) space**. This dual representation is very useful in simulations.

> In what follows, we'll use the shorthand notation ρ_k and Φ_k to denote the discrete Fourier-transformed fields, corresponding to $\hat{\rho}(\mathbf{k})$ and $\hat{\Phi}(\mathbf{k})$ in the continuous case, but defined only at the discrete $\mathbf{k}$ values of our simulation grid.

The Convolution Theorem This theorem is the heart of the entire PM method. It states:

> A slow and complicated **convolution** in real space becomes a fast and simple element-by-element **multiplication** in frequency space.

This allows us to replace the slow, brute-force calculation with a faster three-step process:

1. **Transform to Frequency Space:** We use the **FFT** to transform our mass grid, ρ, into its frequency representation, ρ_k. A convenient property of the FFT is that it automatically treats the finite grid as if it were periodic. It represents the data as a sum of simple waves that fit perfectly end-to-end within the box, which is mathematically equivalent to assuming the grid repeats infinitely like a tiled pattern.

   To compute the gravitational potential, we also need the transform of the function that describes how a unit mass influences space—the **Green's function**. Formally, the Green's function, $G(\mathbf{r})$, is defined as the solution to Poisson's equation for a unit point source:

   $$\nabla^2 G(\mathbf{r}) = 4\pi G\delta(\mathbf{r}).$$

   Where $\delta(\mathbf{r})$ is the Dirac delta function, which represents an idealized point source. In three dimensions, this gives $G(\mathbf{r}) = -G/|\mathbf{r}|$. This mathematical kernel acts as the system's response to a point mass, linking the density field to the potential through Poisson's equation.

   When we move to frequency space, derivatives become multiplications by $-k^2$, so the corresponding frequency-space form of the Green's function is

   $$\mathcal{F}\{\nabla^2 G(\mathbf{r})\} = -k^2 G_k$$

   $$\mathcal{F}\{4\pi G\delta(\mathbf{r})\} = 4\pi G$$

   $$G_k = -\frac{4\pi G}{k^2}.$$

   This is the function we use to compute the potential in the Fourier domain. This function has a mathematical singularity at the $k = 0$ **mode**. This mode (also known as the DC component) represents the **average potential** of the entire simulation box.

   However, since physical forces depend only on the **gradient** of the potential ($\mathbf{F} = -\nabla\Phi$), not its absolute value, this average potential is physically arbitrary. To avoid the division-by-zero, we can redefine the $k = 0$ component of the potential to zero. Beyond just fixing a numerical error, this is mathematically equivalent to subtracting the mean background density of the universe from the source term. This is a standard technique in cosmology (often related to the "Jeans swindle"), which ensures that gravity is driven only by local *perturbations* (overdensities and underdensities) rather than the infinite mass of a periodic universe. It makes the calculation perfectly well-defined without affecting the final relative forces.

2. **Multiply:** In frequency space, the Poisson equation becomes an element-wise multiplication:

   $$\Phi_k = G_k \cdot \rho_k$$

3. **Transform Back:** We take the resulting potential in frequency space, Φ_k, and use the **Inverse Fast Fourier Transform (IFFT)** to convert it back into the real-space potential grid, Φ, that we wanted.

By taking this detour through frequency space, we replace a slow algorithm that scales as $O(M^6)$ (for M grid cells) with one that scales as $O(M^3 \log M)$. This is what makes the Particle-Mesh method convenient, enabling simulations with millions or billions of particles.

Step 2. From Potential to Force

Now that we have the potential grid, Φ, we can calculate the force grid. The physical relationship is universal: force is the negative gradient of the potential.

$$\mathbf{F} = -\nabla\Phi$$

On a discrete grid, we can't take a true derivative. We approximate it using a **finite difference**. A common and accurate method is the **central difference**, which calculates the slope at a point by looking at the values of its neighbors on either side. For the x-component of the force at grid cell (i, j, k), the formula is:

$$F_{x,i,j,k} \approx -\frac{\Phi_{i+1,j,k} - \Phi_{i-1,j,k}}{2L}$$

With the force calculated at every point on the grid, the final step is to **interpolate** this force back to each particle's continuous position. This is done using the same scheme we used for mass assignment (e.g., NGP or CIC), completing the Particle-Mesh calculation.

Key Literature & Further Reading
Breton, Michel-Andrès. (2024). *PySCo: A fast Particle-Mesh N-body code for modified gravity simulations in Python.* arXiv:2410.20501. Available at https://export.arxiv.org/abs/2410.20501

Advanced Interpolation

The Flaws of Nearest Grid Point (NGP)

In a previous section, we introduced the Nearest Grid Point (NGP) scheme as the simplest way to assign mass to the grid. While its simplicity is appealing, it comes at a significant cost to the simulation's accuracy. The "blocky," pixelated density field it creates leads to an equally blocky and unphysical force field.

The primary flaw of NGP is that the force a particle feels is **discontinuous**. A real gravitational field is smooth and changes continuously with position. The jerky, stepwise force from an NGP grid is a poor and unphysical approximation. This leads to several significant problems:

1. **Poor Energy Conservation:** This is the most damaging consequence. Symplectic integrators like Velocity Verlet can only conserve energy if the force is the smooth gradient of a potential. The sudden "jumps" in force at the cell boundaries introduce small, systematic errors into the integration. These errors accumulate over time, causing the total energy of the simulation to **drift** upwards or downwards, rather than just oscillating around the true value.
2. **Violation of Newton's Third Law:** While the total momentum of the system may be globally conserved, the force between any specific pair of particles is not guaranteed to be equal and opposite. The force on particle A depends only on which cell it's in, and the force on particle B depends only on which cell *it's* in. This crude mediation by the grid breaks the pairwise symmetry required for good energy conservation.
3. **Grid-Imposed Artifacts:** The force field has an artificial, grid-like pattern. Particles can feel an unphysical pull along the grid axes (x and y) that is stronger than the pull along the diagonals. This can cause particles to artificially cluster along grid lines, a distracting and inaccurate artifact of the method.

Because of these flaws, NGP is rarely used in simulations where accuracy is a priority. To achieve the stable, energy-conserving behavior we need, we must adopt a smoother method for connecting the particles to the grid, which leads us to the Cloud-in-Cell scheme.

Cloud-in-Cell (CIC)

To achieve a stable simulation that conserves energy, we need a smooth way to connect the particles to the grid. The standard method for this is the **Cloud-in-Cell (CIC)** interpolation scheme.

Particles as Clouds

Instead of treating each particle as an infinitesimal point, the CIC method imagines each particle as a small, **cubic "cloud"** of mass, the same size as a grid cell. As this particle-cloud moves through the simulation space, it naturally overlaps with the **eight** nearest grid points that form the corners of the cell it currently occupies.

The mass of the particle is then distributed, or "splatted," onto these eight grid points. The amount of mass assigned to each point is simply proportional to the **volume of overlap** between the particle's cloud and the region surrounding each grid point. This is a form of **trilinear interpolation**. A particle in the exact center of a cubic cell would distribute 12.5% of its mass to each of the eight corners. A particle mostly in one corner of a cell would give most of its mass to that corner's node.

This process results in a much smoother and more physically realistic mass density grid. A small movement by a particle leads to a small, continuous change in the mass distribution on the grid, completely eliminating the sudden "jumps" of the NGP method.

At its core, CIC is a linear interpolation scheme. Higher-order schemes (like TSC or PCS) exist and provide smoother forces, but are not in the scope of this text.

Symmetric Interpolation

A smooth mass distribution is only half the story. The true magic of CIC, and the reason it conserves energy, lies in its perfect symmetry.

After the forces have been calculated on the grid, we must interpolate them back to the particle's continuous position. The rule for energy conservation is that the **force interpolation scheme must be consistent with the mass assignment scheme**.

CIC follows this rule perfectly. The force on the particle is calculated by taking a weighted average of the forces from the **same eight grid points**, using the **exact same volume-based weights** that were used to distribute the mass.

This symmetry between "splatting" the mass and "gathering" the force ensures that Newton's third law ($\mathbf{F}_{ij} = -\mathbf{F}_{ji}$) is precisely obeyed for any pair of particles, even though their interaction is being mediated by the grid. Because the forces are perfectly reciprocal, the force field is numerically conservative.

When this conservative force is fed into a symplectic integrator, the system's total energy is conserved remarkably well. The systematic energy *drift* seen with NGP is transformed into the small, bounded *oscillation* expected from a high-quality simulation. While slightly more complex to implement, the improvement in accuracy and stability makes CIC the standard choice for most modern Particle-Mesh codes.

Implementation: "Splatting" Mass and "Gathering" Forces

The conceptual idea of treating particles as "clouds" translates into a clean, two-part algorithm. In simulation jargon, these two parts are often called **"splatting"** (distributing the particle mass onto the grid) and **"gathering"** (interpolating the force from the grid back to the particle).

The key to energy conservation is that these two operations must be perfectly symmetric, using the exact same weights for both processes.

For simplicity, the following explanation is for a 2D case.

The "Splat" Step: Mass Assignment

This process is performed for every particle to create the final mass density grid, ρ.

1. Find the Reference Grid Point and Fractional Position

First, for a given particle p with position $\mathbf{x}_p = (x_p, y_p)$, we find the integer index of the grid point at its "bottom-left," denoted (i, j). We then find the particle's fractional position within that cell, (δ_x, δ_y), where both values range from 0 to 1. Let L be the side length of a grid cell.

The reference grid index is found using the floor function, $\lfloor \cdot \rfloor$:

$$i = \lfloor x_p/L \rfloor$$

$$j = \lfloor y_p/L \rfloor$$

The fractional position within the cell is then:

$$\delta_x = (x_p/L) - i$$

$$\delta_y = (y_p/L) - j$$

2. Calculate the Area Weights

Next, we calculate four weights based on these fractional positions. Each weight corresponds to the fractional area of the particle's "cloud" that overlaps with the four surrounding grid cells located at (i, j), $(i + 1, j)$, $(i, j + 1)$, and $(i + 1, j + 1)$.

The weights for the corners are:

$$w_{i,j} = (1 - \delta_x)(1 - \delta_y)$$

$$w_{i+1,j} = \delta_x(1 - \delta_y)$$

$$w_{i,j+1} = (1 - \delta_x)\delta_y$$

$$w_{i+1,j+1} = \delta_x\delta_y$$

Notice that these four weights always sum to 1.

3. Distribute the Mass

Finally, we add the mass of the particle, m_p, scaled by the appropriate weight, to each of the four corresponding grid points. This process is repeated for all particles in the simulation. The total mass on a given grid point is the sum of the contributions from all particles whose "clouds" overlap it.

The contribution from a single particle p to the mass at each of the four nodes is:

$$\Delta M_{i,j} = m_p \cdot w_{i,j}$$

$$\Delta M_{i+1,j} = m_p \cdot w_{i+1,j}$$

$$\Delta M_{i,j+1} = m_p \cdot w_{i,j+1}$$

$$\Delta M_{i+1,j+1} = m_p \cdot w_{i+1,j+1}$$

To get the final mass density grid, ρ, the total mass accumulated at each node is divided by the cell area, L^2. All grid indices are taken modulo the grid size to correctly handle the periodic boundaries.

The "Gather" Step: Force Interpolation

This step occurs after the forces have been calculated on the grid (creating an acceleration field, $\mathbf{a}_{\text{grid}}$) and is the mirror image of the splatting process.

To find the force on a particle, we use the **exact same** indices and weights we would calculate for it in the splatting step. We then perform a weighted average of the acceleration values from the four surrounding grid points to find the acceleration at the particle's precise location, $\mathbf{a}_p$.

Let the acceleration field on the grid be $\mathbf{a}_{i,j} = (a_{x,i,j}, a_{y,i,j})$ and the four CIC weights for a given particle be $w_{i,j}$, $w_{i+1,j}$, $w_{i,j+1}$, and $w_{i+1,j+1}$.

The x-component of the interpolated acceleration for the particle, $a_{x,p}$, is calculated as:

$$a_{x,p} = a_{x,i,j} \cdot w_{i,j} + a_{x,i+1,j} \cdot w_{i+1,j} + a_{x,i,j+1} \cdot w_{i,j+1} + a_{x,i+1,j+1} \cdot w_{i+1,j+1}$$

The y-component, $a_{y,p}$, is calculated in the exact same way using the y-components of the grid acceleration field.

The final force on the particle, $\mathbf{F}_p$, is its mass, m_p, times this interpolated acceleration vector:

$$\mathbf{F}_p = m_p\mathbf{a}_p$$

This symmetric "Splat-Gather" procedure ensures that the forces are numerically conservative, which is the fundamental reason why CIC allows a symplectic integrator to conserve energy over long periods.

Key Literature & Further Reading

Bagla, J. S., & Padmanabhan, T. (2004). *Cosmological N-Body Simulations.* arXiv:astro-ph/0411730. Available at https://arxiv.org/pdf/astro-ph/0411730.pdf

The P³M Algorithm

Combining PP for Short-Range and PM for Long-Range

We have seen that the Particle-Mesh (PM) method is efficient for calculating the gravitational field of a large number of particles. However, its speed comes at the cost of accuracy at small scales. The grid is good at capturing the overall "blurry" shape of the gravitational field, but it's innacurate at resolving the sharp, fine details of the force between two particles that are very close to each other. This inaccuracy at short ranges is the primary weakness of the pure PM method.

On the other hand, the direct Particle-Particle (PP) calculation is the exact opposite. While it is perfectly accurate at all scales, its weakness, is that its $O(N^2)$ complexity makes it too slow for a large number of particles.

This presents a classic trade-off: speed or accuracy. The **Particle-Particle Particle-Mesh (P³M)** algorithm provides an elegant solution by combining both methods, using each one only where it excels.

The P³M method splits the force calculation into two parts based on a **cutoff radius**, r_c:

1. **Long-Range Force (PM):** The smooth, gentle pull from all the **distant** particles (those farther than r_c) is calculated efficiently using the Particle-Mesh method.
2. **Short-Range Force (PP):** The sharp, strong force from the few **nearby** particles (those closer than r_c) is calculated precisely using the direct Particle-Particle method.

The Subtractive Scheme

Simply adding these two forces together would be incorrect, as the PM method already includes an inaccurate estimate of the short-range forces. Instead, we use the PP calculation to *correct* the PM force at short distances. This is often done with a **subtractive scheme**:

$$\mathbf{F}_{\text{total}} = \mathbf{F}_{\text{PM}} + \left(\mathbf{F}_{\text{PP}}^{\text{short}} - \mathbf{F}_{\text{PM}}^{\text{short}}\right)$$

The process is straightforward:

1. First, we calculate the baseline $\mathbf{F_{PM}}$ for all particles. This gives us the correct long-range force everywhere but an incorrect "blurry" force for nearby pairs.
2. Then, for any pair of particles closer than the cutoff radius, we calculate the **true, sharp force** between them, $\mathbf{F}_{\mathbf{PP}}^{\mathbf{short}}$.
3. We also calculate an approximation of the **blurry, inaccurate force** that the PM method produced for that same pair, $\mathbf{F}_{\mathbf{PM}}^{\mathbf{short}}$.
4. Finally, we subtract the inaccurate mesh force and add the correct direct force. This effectively replaces the blurry grid force with the sharp, accurate PP force, but only where it matters—at short distances.

By dividing the problem this way, P³M leverages the strengths of both methods. It uses the fast PM algorithm for the vast majority of interactions (the thousands of weak pulls from distant particles) and reserves the slow but accurate PP algorithm only for the few critical interactions between close neighbors. The result is a simulation that is nearly as fast as a pure PM code but nearly as accurate as a pure PP code—the true best of both worlds.

Calculating the Mesh-Force Correction

To implement the subtractive scheme, we need a mathematical function for $\mathbf{F}_{\text{PM}}^{\text{short}}$ that approximates the "blurry" force produced by the grid at short distances. We can't get this from the final grid itself, as it contains the combined influence of all particles.

Instead, we model this effect with a standard gravitational force formula that has been **softened** with a special parameter, ϵ_{PM}, chosen specifically to mimic the resolution of the Particle-Mesh grid.

The vector force that approximates the mesh's influence between two particles with masses m_1 and m_2 is given by:

$$\mathbf{F}_{\text{PM}}^{\text{short}} = \frac{G m_1 m_2 \mathbf{r}}{\left(r^2 + \epsilon_{\text{PM}}^2\right)^{3/2}}$$

The terms in this formula are:

- $\mathbf{r}$ is the vector separating the two particles.

- r is the magnitude of that vector, $r = \|\mathbf{r}\|$.
- G, m_1, m_2 are the gravitational constant and the particle masses.
- ϵ_{PM} is the crucial term: a **softening length** specifically chosen to match the grid's resolution. A standard and effective choice is to set this value to be proportional to the grid cell length, L. For example:

$$\epsilon_{\text{PM}} \approx 0.5 \cdot L$$

This formula creates a force that is significantly weakened at short distances (when $r \lesssim \epsilon_{\text{PM}}$), which successfully mimics the behavior of the full PM/FFT calculation. By subtracting this specific force in the correction step, we effectively cancel out the grid's primary error at short range.

Choosing the Cutoff Radius (r_c)

The choice of the cutoff radius, r_c, is a crucial tuning parameter in a P³M simulation. It represents a trade-off between accuracy and computational speed.

- A **small** cutoff radius means the fast PM method handles most of the work, but we risk losing accuracy if the cutoff is smaller than the region where the PM force is unreliable.
- A **large** cutoff radius ensures high accuracy at short ranges, but it forces the slow PP calculation to do much more work, which can bog down the entire simulation.

The optimal choice is not arbitrary; it is fundamentally linked to the resolution of the Particle-Mesh grid. The PM method's accuracy degrades significantly at distances smaller than about 2 to 3 grid cell sizes. Therefore, the cutoff radius must be large enough to ensure the accurate PP method is used throughout this entire "inaccurate zone."

A standard and robust rule of thumb is to set the cutoff radius to be a few times the grid cell length, L:

$$r_c \approx 2.5 \cdot L$$

This choice guarantees that the sharp, correct PP force is used wherever the PM force is most likely to fail. The primary parameter that is usually tuned is the grid size itself; once the grid size is chosen, the cutoff radius is set accordingly to maintain this balance.

The Switching Function

The subtractive scheme is a powerful way to correct for the short-range errors of the Particle-Mesh method. However, using a hard cutoff radius—where the correction is fully active if $r < r_c$ and instantly zero if $r \geq r_c$—can create a small, abrupt "jolt" in the force. This discontinuity, however small, can introduce numerical errors and impact the long-term energy conservation of the simulation.

To achieve the highest accuracy, we must ensure the total force is a perfectly smooth function at all distances. This is accomplished by introducing a **switching function**, $S(r)$, that smoothly "fades out" the short-range correction as the particle separation, r, approaches the cutoff radius, r_c.

The total force is then calculated as:

$$\mathbf{F}_{\text{total}} = \mathbf{F}_{\text{PM}} + S(r) \cdot \left(\mathbf{F}_{\text{PP}}^{\text{short}} - \mathbf{F}_{\text{PM}}^{\text{short}}\right)$$

The switching function $S(r)$ operates over a small **transition zone**, typically defined between a starting radius, r_{start}, and the cutoff radius, r_c. It has the following properties:

1. For $r \leq r_{\text{start}}$, the function is $S(r) = 1$. The correction is fully applied.
2. For $r \geq r_c$, the function is $S(r) = 0$. The correction is fully turned off.
3. In the transition zone, $r_{\text{start}} < r < r_c$, the function smoothly decreases from 1 to 0.

To ensure the force changes perfectly smoothly, the *derivative* of the switching function should also be zero at the start and end of the transition. A standard and effective way to achieve this is with a cubic polynomial.

First, we define a normalized distance, x, that goes from 0 to 1 across the transition zone:

$$x = \frac{r - r_{\text{start}}}{r_c - r_{\text{start}}}$$

Then, a polynomial that satisfies all the smoothness conditions is:

$$S(x) = 2x^3 - 3x^2 + 1$$

Using this function to taper the correction term eliminates the unphysical jolt at the cutoff. It creates a continuous and differentiable total force, which allows the symplectic integrator to perform optimally and leads to superior long-term energy conservation.

Key Literature & Further Reading
Shirokov, A., & Bertschinger, E. (2005). *GRACOS: Scalable and Load Balanced P3M Cosmological N-body Code.* arXiv:astro-ph/0505087. Available at https://arxiv.org/abs/astro-ph/0505087

An Expanding Space

Up to this point, our simulation has taken place in a static box. This is a good approximation for a star cluster or a single galaxy, but it is fundamentally wrong for a cosmological simulation. Our universe is not static; it is expanding. To accurately model the formation of structure, we must incorporate this expansion into our simulation.

This is achieved by switching from familiar "proper" coordinates to a more abstract but powerful system called **comoving coordinates**. Instead of tracking particles in a fixed box, we track them on a virtual grid that expands along with the universe itself.

The Hubble Flow

The dominant motion in the universe is the cosmic expansion, a phenomenon described by the **Hubble-Lemaître Law**. This law states that, on average, every galaxy is moving away from every other galaxy. The farther away a galaxy is, the faster it appears to recede. This is not a motion *through* space, but rather the expansion *of* space itself. This uniform expansion is the **Hubble Flow**. The velocity of this recession, $\mathbf{v}_{\text{Hubble}}$, for an object at a proper distance $\mathbf{r}$ is given by:

$$\mathbf{v}_{\text{Hubble}}(t) = H(t)\mathbf{r}(t)$$

Where $H(t)$ is the Hubble parameter at time t. This flow is the background upon which all other motions are superimposed.

Comoving Coordinates

Tracking particles whose primary motion is this rapid expansion is computationally difficult. It's much easier to factor out the expansion. We do this by defining a **scale factor**, $a(t)$, which describes the relative size of the universe at any time t. By convention, $a = 1$ today. In the past, a was smaller.

We can now define two types of coordinates:

- **Proper Coordinates (r):** The real, physical distance between two objects that you would measure with a ruler at time t. This distance grows as the universe expands.
- **Comoving Coordinates (x):** The coordinates of an object on our virtual, expanding grid. If an object is moved *only* by the Hubble Flow, its comoving coordinates **do not change**.

The relationship between them is simple:

$$\mathbf{r}(t) = a(t)\mathbf{x}$$

A particle's true velocity is a combination of the Hubble Flow and its own, small-scale motion relative to the expanding grid. This local motion, caused by the gravitational pull of nearby structures, is called the **peculiar velocity**, $\mathbf{v}_{\text{pec}}$.

The Equations of Motion

To understand how the equations of motion change, we start with the standard physical law in **proper coordinates**: a particle's physical acceleration is equal to the true gravitational acceleration at its location.

$$\frac{d^2\mathbf{r}}{dt^2} = \mathbf{g}_{\text{proper}}(\mathbf{r})$$

Here, $\mathbf{g}_{\text{proper}}(\mathbf{r})$ is the "real" gravitational acceleration created by the physical distribution of matter in the expanding universe.

Our goal is to rewrite this equation using **comoving coordinates**, $\mathbf{x}$, which are related by $\mathbf{r}(t) = a(t)\mathbf{x}$. After performing the necessary calculus to account for the fact that the scale factor $a(t)$ is changing over time, we arrive at the new equation of motion for a particle's comoving acceleration, $\frac{d^2\mathbf{x}}{dt^2}$:

$$\frac{d^2\mathbf{x}}{dt^2} = \frac{\mathbf{g}_{\text{comoving}}(\mathbf{x})}{a^3} - 2H(t)\frac{d\mathbf{x}}{dt}$$

Let's break down these two terms, which are the fundamental modifications needed for a cosmological simulation.

- **Modified Gravity:** The first term, $\frac{\mathbf{g}_{\text{comoving}}(\mathbf{x})}{a^3}$, represents the force of gravity. The term $\mathbf{g}_{\text{comoving}}(\mathbf{x})$ is the gravitational acceleration that our force solvers (like PM or PP) calculate—it's the acceleration that would exist in a *static* universe if the particles were at their current comoving positions. The division by the scale factor cubed, a^3, is the crucial cosmological correction. As the universe expands by a factor of a, the volume of any given region increases by a^3. This dilutes the physical density of matter as $\rho \propto 1/a^3$. Since gravity is sourced by density, its strength weakens accordingly, and this term correctly captures that effect.
- **Hubble Drag:** The second term, $-2H(t)\frac{d\mathbf{x}}{dt}$, is a new velocity-dependent "friction" term. The term $H(t)$ is the Hubble parameter ($\frac{1}{a}\frac{da}{dt}$), and $\frac{d\mathbf{x}}{dt}$ is the particle's **peculiar velocity** (its local motion relative to the expanding grid). This "Hubble drag" acts to slow down these peculiar velocities. In an expanding universe, a particle's local motion is constantly being damped by the stretching of space itself.

By solving this new equation of motion, our simulation correctly captures the delicate interplay between the cosmic expansion that tries to pull everything apart and the force of gravity that tries to pull everything together.

Cosmological Models and the Friedmann Equations

The equation of motion we just derived tells us how particles respond to gravity and expansion, but it relies on two crucial background variables: the scale factor, $a(t)$, and the Hubble parameter, $H(t)$. To actually integrate the particle trajectories, our simulation needs to know exactly how these values evolve. Their behavior is not arbitrary; it is dictated by the fundamental laws of General Relativity.

General Relativity is governed by **Einstein's Field Equations**. At their core, these equations describe the delicate balance between the geometry of the cosmos and the "stuff" inside it. They are elegantly summarized in tensor notation:

$$G_{\mu\nu} + \Lambda g_{\mu\nu} = \frac{8\pi G}{c^4}T_{\mu\nu}$$

In this formulation, the left side represents the canvas of spacetime itself: $G_{\mu\nu}$ measures the geometric curvature, $g_{\mu\nu}$ is the metric defining how distances are measured, and Λ is the cosmological constant representing the inherent energy of the vacuum. The right side represents the contents: $T_{\mu\nu}$ is the stress-energy tensor, which tallies up all the mass, light, and fluid pressure in a given region. As physicist John Archibald Wheeler famously summarized: *"Spacetime tells matter how to move; matter tells spacetime how to curve."* However, because this compact line actually hides ten grueling, interconnected differential equations, solving it directly for an irregular, clumpy universe filled with billions of scattered galaxies is mathematically impossible.

In the 1920s, physicist Alexander Friedmann simplified Einstein's field equations for a universe that is assumed to be uniform and isotropic on large scales. The resulting **Friedmann equation** acts as the master blueprint for cosmic expansion. Mathematically, it relates the universe's expansion rate to its matter density, geometric curvature, and the cosmological constant:

$$H(t)^2 = \left(\frac{1}{a}\frac{da}{dt}\right)^2 = \frac{8\pi G}{3}\rho - \frac{kc^2}{a^2} + \frac{\Lambda c^2}{3}$$

Here, G is the gravitational constant, ρ represents the density of matter and radiation, k is a constant representing the overall geometric curvature of space, and Λ (Lambda) is the cosmological constant—a term representing the inherent energy of the vacuum itself.

Observations of the real universe strongly indicate that our cosmos is geometrically "flat," meaning $k = 0$. This simplifies the equation significantly.

A **cosmological model** is simply a specific "recipe" of these cosmic ingredients. By defining what our virtual universe is made of (the amount of matter ρ and vacuum energy Λ) and plugging them into the Friedmann equation, we can mathematically solve for the exact historical trajectory of the expansion.

For the purposes of our simulation, there are two primary models of interest: the classic, matter-dominated model (Einstein-de Sitter) and the modern, dark-energy-driven model (ΛCDM).

An Einstein-de Sitter Universe

For a simulation to be physically meaningful, it must be based on a self-consistent cosmological model. The simplest and most classic model for a matter-dominated universe is the **Einstein-de Sitter (EdS)** model. This is a specific solution to Einstein's Friedmann equations that describes a flat, expanding universe containing only matter and no cosmological constant:

$$H(t)^2 = \frac{8\pi G}{3}\rho(t)$$

In an EdS universe, the expansion of space is constantly being decelerated by gravity. This physical reality is described by a simple power-law relationship between the scale factor, a, and cosmic time, t:

$$a(t) \propto t^{2/3}$$

From this, the Hubble parameter,

$$H(t) = \frac{1}{a(t)}\frac{da(t)}{dt},$$

is also a simple function of time:

$$H(t) = \frac{2}{3t}$$

Critical Density and Model Consistency

As demonstrated by the Friedmann equation, if we define a universe with a flat geometry ($k = 0$) and no cosmological constant ($\Lambda = 0$), the expansion rate is perfectly balanced by the density of the universe. This specific equilibrium point is known as the **critical density**, $\rho_c(t)$. In a universe without dark energy, it represents the precise density required to halt the cosmic expansion after an infinite amount of time, defined entirely by the Hubble parameter and the gravitational constant, G:

$$\rho_c(t) = \frac{3H(t)^2}{8\pi G}$$

Because an Einstein-de Sitter universe perfectly matches these exact conditions—it is flat and contains exclusively matter—its average **matter density** must equal this critical density at all times. For our simulation to be a consistent representation of this model, the mean density of our simulation box—the total mass, M_{total}, divided by the proper (physical) volume, $V = (aL)^3$—must also satisfy this requirement:

$$\rho_{mean}(t) = \frac{M_{total}}{(a(t)L)^3}$$

By equating $\rho_{mean} = \rho_c$ and substituting the EdS relations for $a(t)$ and $H(t)$, we find that the simulation parameters are not independent but must be linked by a **consistency relation**.

Natural Units

To simplify the implementation, it is standard practice to work in a system of **natural units** where key quantities are set to 1. A common choice for N-body simulations is to set:

- The total mass of the system: $M_{total} = 1$
- The comoving side length of the box: $L = 1$
- The present-day scale factor: $a(t_{today}) = 1$

With these choices, the consistency relation is no longer a check but is used to *define* the value of the gravitational constant required for the simulation. Equating the mean and critical densities in this unit system fixes the value of G:

$$G = \frac{3H(t)^2(a(t)L)^3}{8\pi M_{total}} = \frac{L^3}{6\pi M_{total}} = \frac{1}{6\pi}$$

By using this specific value for G, we ensure that the strength of gravity in our simulation is perfectly balanced against the expansion rate, allowing for the realistic, hierarchical growth of structure.

The ΛCDM Model and Dark Energy

The Einstein-de Sitter (EdS) model is mathematically elegant and perfectly describes a universe dominated entirely by the gravity of matter. For decades, it was the standard model of cosmology. However, in 1998, observations of distant supernovae revealed a shocking truth: the expansion of our universe is not slowing down due to gravity; it is accelerating.

To model the real universe, we must upgrade from EdS to the **ΛCDM (Lambda Cold Dark Matter)** model. This model introduces a new component to the cosmos: **Dark Energy**, represented by the cosmological constant, Λ. Dark energy acts as a repulsive negative pressure inherent to space itself, pushing the universe apart.

In a flat ΛCDM universe, the total density is made up of matter ($\Omega_m \approx 0.3$) and dark energy ($\Omega_\Lambda \approx 0.7$), such that $\Omega_m + \Omega_\Lambda = 1$. The Friedmann equation expands to include this new term:

$$H(t)^2 = H_0^2 \left(\frac{\Omega_m}{a(t)^3} + \Omega_\Lambda \right)$$

Where H_0 is the **Hubble Constant**—the exact rate at which the universe is expanding right now, that is, the value of the Hubble Parameter ($H(t)$) measured at the present.

Notice that the matter density dilutes as the universe expands ($1/a^3$), but the dark energy density (Ω_Λ) remains perfectly constant. This creates a fascinating cosmic tug-of-war.

The Evolution of the Scale Factor

In the early universe, when $a(t)$ was very small, the matter term completely dominated the Friedmann equation. The universe behaved almost exactly like an EdS universe, decelerating as gravity pulled matter together. However, as space expanded and matter diluted, the constant push of dark energy eventually overtook the fading pull of gravity. Today, dark energy dominates, and the expansion is accelerating.

The exact solution for the scale factor $a(t)$ in a flat ΛCDM universe gracefully captures both of these eras—the early deceleration and the late acceleration—using a hyperbolic sine function:

$$a(t) = \left(\frac{\Omega_m}{\Omega_\Lambda} \right)^{1/3} \sinh^{2/3} \left(\frac{3}{2} H_0 \sqrt{\Omega_\Lambda} t \right)$$

By using this equation, along with its corresponding Hubble parameter $H(a)$, our simulation smoothly transitions from the matter-dominated epoch (where structures rapidly form) into the dark-energy-dominated epoch (where the cosmic web is stretched and frozen in place).

Dark energy actively fights against the formation of galaxies. In a pure matter universe, structures grow steadily. But in a ΛCDM universe, the accelerated stretching of space pulls matter apart faster than gravity can pull it together.

Natural Units in a ΛCDM Universe

Because the ΛCDM model also describes a geometrically flat universe, its total energy density must still equal the critical density, ρ_c. However, in our simulation, the particles only represent matter; they do not represent the vacuum energy. Therefore, the mean density of our computational grid accounts only for the matter fraction: $\bar{\rho}_m = \Omega_m \rho_c$.

At first glance, it seems like we need to modify our gravitational constant, G, to account for this diluted matter fraction. However, the elegance of our natural unit system reveals a beautiful mathematical cancellation.

The critical density is defined by the present-day Hubble parameter, H_0:

$$\rho_c = \frac{3H_0^2}{8\pi G}$$

In our simulation units, H_0 is intrinsically linked to the matter density to ensure the correct timeline for cosmic expansion, defined mathematically as $H_0 = \frac{2}{3\sqrt{\Omega_m}}$. If we plug this into our equation for the mean matter density, we get:

$$\bar{\rho}_m = \Omega_m \left(\frac{3\left(\frac{2}{3\sqrt{\Omega_m}}\right)^2}{8\pi G} \right)$$

When we expand the squared Hubble term, the Ω_m in the denominator perfectly cancels out the Ω_m scaling the equation:

$$\bar{\rho}_m = \Omega_m \left(\frac{3\left(\frac{4}{9\Omega_m}\right)}{8\pi G} \right) = \frac{12}{72\pi G} = \frac{1}{6\pi G}$$

The mean density of our simulation box is defined by the total mass, M_{total}, divided by its volume, L^3. Equating our grid density to the physical matter density gives us our final equation for G:

$$\frac{M_{total}}{L^3} = \frac{1}{6\pi G}$$

$$G = \frac{L^3}{6\pi M_{total}}$$

Remarkably, the Ω_m parameter completely drops out of the calculation for G. Because the definition of our expansion rate (H_0) already absorbs the matter fraction, the natural units derived for the simple Einstein-de Sitter model remain perfectly valid and exact for a complex ΛCDM universe.

Hubble: Physical vs. Code Units

When looking at the mathematical derivation above, a sharp reader might spot an apparent contradiction. We defined the present-day Hubble parameter (the Hubble constant) mathematically as $H_0 = \frac{2}{3\sqrt{\Omega_m}}$. However, in observational cosmology, the matter density (Ω_m) and the Hubble constant (H_0) are completely independent parameters. You can physically have a universe where $\Omega_m = 0.3$ and $H_0 = 70$ km/s/Mpc, or one where $\Omega_m = 1.0$ and $H_0 = 50$ km/s/Mpc.

So, why does our simulation mathematically force them to be linked?

The answer lies in our choice of **dimensionless code units**. Conceptually, H_0 always represents the exact same thing: the present-day expansion rate of the universe. However, because our simulation strips away standard physical units, we lose the ability to measure speed in kilometers or time in seconds.

Specifically, we set our box length to $L = 1$ and our total mass to $M_{total} = 1$ (which permanently locks our average density at $\bar{\rho}_0 = 1$). Furthermore, we scaled our gravitational constant to exactly $G = \frac{1}{6\pi}$. When you plug these dimensionless code values into the standard real-world Friedmann equation,

$$\Omega_m = \frac{8\pi G \bar{\rho}_0}{3H_0^2}$$

the constants mathematically collapse, yielding our exact internal equation: $H_0 = \frac{2}{3\sqrt{\Omega_m}}$.

To remain physically accurate, a robust simulation must handle the translation between the physical world and this internal code grid by splitting the parameter into two roles:

- **The Physical Hubble Parameter (h):** In observational astronomy, the Hubble constant is typically written as $H_0 = 100 \cdot h$ **km/s/Mpc**, where h is a dimensionless scaling factor (often around 0.7). This parameter represents the real-world expansion speed. It is used during the setup phase to generate the initial conditions, calculating the true physical size of the primordial density fluctuations and converting physical gas temperatures into internal energy.
- **The Internal Code-Unit Hubble Parameter:** Once the simulation starts running, it stops thinking in kilometers and seconds. Because we fixed our grid's mass, volume, and gravity to arbitrary values, the collapsed Friedmann equation ($H_0 = \frac{2}{3\sqrt{\Omega_m}}$) calculates the exact internal "ticking rate" our code clock requires to ensure the gravitational collapse of our dimensionless mass perfectly balances the expansion of our grid.

In practice, a simulation isolates these two systems. The time integrator uses the internal code-unit H_0 to stretch the scale factor $a(t)$ and apply the Hubble drag to the particles natively on the grid. Meanwhile, the physical h parameter is used to shape the initial conditions and translate the final outputs back into physical miles, kilograms, and Kelvin. By managing this unit conversion, the code remains mathematically stable while allowing the user to input any combination of Ω_m and h they choose.

The Cosmic Timeline

The Scale Factor (a) and Redshift (z)

Before we look at the timeline of the universe, we need to define how cosmologists actually tell time. While it is intuitive to talk about "years since the Big Bang," it is incredibly difficult to calculate. Instead, astronomers rely on two interlocked variables to track the evolution of the cosmos: the scale factor (a) and cosmological redshift (z).

The Scale Factor (a) As we established, $a(t)$ is the mathematical engine of the universe's expansion. It tracks the relative, physical size of the coordinate grid. By convention, the scale factor today is set to exactly $a = 1$. When the universe was a quarter of its current size, $a = 0.25$. As we run our code forward from the early universe, a continuously ticks upward toward 1.

Cosmological Redshift (z) While a is perfect for writing computer code, an astronomer cannot point a telescope at the sky and directly measure the "size of the coordinate grid." What they *can* measure is light.

As a photon travels across the universe to reach our telescopes, the space it is traveling through is expanding. This expansion physically stretches the photon's wavelength, shifting its color toward the red end of the spectrum. We call this stretching **Cosmological Redshift**, denoted by the letter z.

The Mathematical Link Because the stretching of the light is tied exactly to the stretching of space itself, redshift and the scale factor are inversely related by a very simple, but universally important equation:

$$a = \frac{1}{1+z}$$

Because redshift is the actual, tangible thing we measure in the real world, it has become the universal "clock" for the history of the universe. In literature we will almost always see time denoted by z:

- $z = 0$**:** Today ($a = 1$).
- $z = 1$**:** The universe was exactly half its current size ($a = 0.5$).
- $z = 9$**:** The universe was 1/10th its current size ($a = 0.1$).
- $z = 49$**:** A typical starting time for generating simulation's initial conditions ($a = 0.02$).
- $z = 1100$**:** The release of the Cosmic Microwave Background ($a \approx 0.0009$).
- $z \to \infty$**:** The Big Bang ($a \to 0$).

When reading cosmological texts, we can think of redshift as a measure of looking backward: the higher the z, the further back in time we are looking, the further away the object is, and the smaller and denser the universe was when that light was emitted.

Eras and epochs

In cosmology, we divide the 13.8-billion-year history of the universe into distinct "eras." These divisions are not arbitrary historical chapters; they are strictly defined by the mathematics of the Friedmann equation.

$$H^2(a) = H_0^2 \left(\Omega_{r,0} a^{-4} + \Omega_{m,0} a^{-3} + \Omega_{k,0} a^{-2} + \Omega_{\Lambda,0} \right)$$

Whichever component of the universe (radiation, matter, or dark energy) has the highest energy density dictates the expansion rate of space and the physical rules for how structures form.

For the purposes of cosmological simulations, the universe's history is defined by three major eras:

The Radiation-Dominated Era (The Big Bang to ~50,000 Years) In the very early universe, space was incredibly small, dense, and unimaginably hot. During this time, the universe's energy budget was completely dominated by the kinetic energy of photons and relativistic particles (neutrinos).

- **The Physics:** Because radiation exerts a massive outward pressure, space expanded very rapidly. This intense photon pressure outpaced the gravitational pull of dark matter, completely stalling the growth of small-scale density fluctuations (the Mészáros effect).
- **Simulation Context:** We rarely simulate this era directly with N-body codes. Instead, its effects are mathematically "baked in" to our initial conditions. The suppression of small-scale structures during this era is exactly what creates the mathematical curve of the **BBKS Transfer Function**.
- **The Transition:** As the universe expanded, radiation diluted much faster than physical matter. Around 50,000 years after the Big Bang, the density of radiation dropped below the density of matter, marking a monumental shift in cosmic physics. Shortly after this transition (at about 380,000 years, or a redshift of

roughly z = 1100), the universe cooled enough for the first neutral atoms to form, releasing the Cosmic Microwave Background (CMB).

The Matter-Dominated Era (~50,000 Years to ~9.8 Billion Years) Once the radiation cleared, the gravity of cold dark matter and baryonic gas took absolute control of the universe. Because matter exerts no large-scale outward pressure, the expansion of the universe began to slow down.

- **The Physics:** This is the golden age of structure formation. With radiation pressure gone and the expansion slowing, gravity was finally free to pull matter together. The tiny primordial ripples left over from the Big Bang collapsed into the cosmic web, forming the first stars, galaxies, and galaxy clusters.
- **Simulation Context:** This era is the primary sandbox for computational cosmology. Our simulations typically start right near the beginning of this era (e.g., at redshift z = 49). Because dark energy is negligible here, the universe behaves like a simple Einstein-de Sitter model where the linear growth factor scales perfectly with the size of the universe: $D(t) \propto a(t)$.
- **Key Epochs:** Inside this era, hydrodynamic simulations track two vital milestones:
  - *The Dark Ages:* The time before the first stars ignited, when the universe was filled with cold, neutral hydrogen gas.
  - *Cosmic Dawn & The Epoch of Reionization:* The violent period when the first massive stars and quasars emitted so much ultraviolet light that they blasted the neutral hydrogen gas back into a hot plasma, fundamentally changing the hydrodynamics of the intergalactic medium.

The Dark Energy-Dominated Era (~9.8 Billion Years to Present) Matter dilutes as the universe expands, but the density of Dark Energy (the cosmological constant, Λ) remains perfectly constant. About 9.8 billion years after the Big Bang (around redshift z = 0.3), the density of matter finally diluted so much that Dark Energy became the dominant force in the cosmos.

- **The Physics:** Dark Energy acts as a repulsive pressure inherent to the vacuum of space. Under its dominance, the expansion of the universe stopped slowing down and began to violently accelerate.
- **Simulation Context:** This late-time acceleration physically stretches the cosmic web apart faster than gravity can pull it together. This marks the epoch where the growth factor $D(t)$ stalls out and stops tracking the scale factor. Large-scale mergers between galaxy clusters become increasingly rare, and the largest structures begin to freeze in place.

Key Literature & Further Reading
Springel, V. (2005). The cosmological simulation code GADGET-2. *Monthly Notices of the Royal Astronomical Society*, 364(4), 1105-1134. Available at: https://arxiv.org/abs/astro-ph/0505010

Scale and Cosmic Variance

Before we can populate our simulation with particles, we must make a fundamental decision: how much of the universe are we going to simulate, and in how much detail?

Because computational resources are finite, choosing the parameters of our simulation requires navigating a strict physical trade-off between the overall size of our simulated box (the macroscopic scale) and the size of our individual grid cells (the microscopic resolution). If we choose poorly, our virtual universe will either fail to form galaxies or fail to represent the actual cosmos.

Cosmic Variance and the Minimum Box Size

Our goal is usually to simulate a "representative" patch of the universe. This means that the statistical properties of our simulation box—the number of galaxy clusters, the sizes of the voids, the web-like structure of the filaments—should look identical to any other randomly selected patch of the real universe of the same size.

However, the universe is only uniform on extremely large scales. If you look at a small patch of space (e.g., 10 Megaparsecs across), you might accidentally center your view on a massive supercluster, or you might look at an entirely empty void. This statistical uncertainty is known as **Cosmic Variance**.

If a simulation box is too small, it suffers from severe cosmic variance. A small box physically cannot contain the longest wavelengths of the density field, meaning it will never form massive superstructures. Furthermore, the periodic boundary conditions will cause the few structures that do form to artificially interact with themselves across the boundaries.

In professional cosmology, the accepted threshold for a simulation volume to be considered statistically representative of the large-scale structure is a comoving box length of roughly **100 Mpc** (Megaparsecs) or larger. At this scale, the simulation volume is vast enough to contain a healthy, statistically average mix of all cosmic environments, from the deepest voids to the most massive cluster nodes.

The Resolution Limit for Halo Formation

While the box must be large enough to capture the cosmic web, the grid cells must be small enough to capture the galaxies within it.

In our Particle-Mesh and P^{3}M algorithms, the grid cell size (L_{cell} = Box Size/Mesh Size) determines the fundamental resolution limit of the simulation. Because forces are smoothed at the scale of the grid cells to ensure numerical stability, the simulation physically cannot form "clumps" of matter that are smaller than a few grid cells across.

In the real universe, the dark matter halos that host standard galaxies (like our Milky Way) have radii on the order of 0.1 to 0.5 Mpc. Therefore, to successfully resolve distinct, tightly collapsed galactic halos, a cosmological simulation requires a spatial resolution of roughly **0.3 to 0.5 Mpc** per cell.

If the resolution is significantly coarser than this (for example, 3.0 Mpc per cell), gravity will still pull matter together, but the small-scale smoothing will prevent sharp collapse. The resulting universe will look "blurry," with matter smeared out into thick filaments rather than forming distinct, highly non-linear galactic nodes.

The Computational Trade-off

These two physical constraints—a minimum box size of 100 Mpc and a target resolution of ~0.4 Mpc—dictate the minimum computational requirements for a realistic simulation.

Because Resolution = Box Size/Mesh Size, achieving a 0.39 Mpc resolution in a 100 Mpc box requires a 3D grid with a mesh size of 256. Simulating this requires 256^3 (roughly 16.7 million) dark matter particles and an equal number of Eulerian gas cells.

This $O(N^3)$ scaling is the harsh reality of 3D hydrodynamics. Doubling the resolution of a simulation requires $2^3 = 8$ times more memory, and vastly more processing time due to the smaller required timesteps.

When testing code or running experiments on smaller machines, it is entirely valid to run smaller "toy models" (for instance, a 24 Mpc box with a 48^3 grid). This perfectly preserves the critical 0.5 Mpc resolution necessary to watch gravity violently collapse halos and shock-heat the gas. However, one must simply keep in mind that such a "dwarf volume" is essentially a zoom-in on a single cosmic neighborhood, sacrificing the grand scale of the cosmic web in exchange for computational speed.

A Reference for Cosmic Scales

Because the Megaparsec (Mpc) is an unfathomably vast unit of distance (1 Mpc $\approx$ 3.26 million light-years), it can be difficult to build an intuition for the scale of a simulation grid. To help anchor these numbers to reality, here is a quick-reference guide to the approximate diameters of common astronomical structures:

Structure	Approximate Diameter (Mpc)	Notes
Earth-Sun Distance (1 AU)	$\sim 5 \times 10^{-12}$ Mpc	The distance light travels in 8 minutes.
The Solar System	~ 0.00001 Mpc	Reaching out to the edge of the Oort Cloud.
Milky Way (Visible Stellar Disk)	~ 0.03 Mpc	The glowing spiral of stars and gas we can see.
Milky Way (Dark Matter Halo)	~ 0.3 Mpc	The invisible gravitational well hosting our galaxy.
The Local Group	~ 3.0 Mpc	Our local neighborhood, including Andromeda.
Typical Galaxy Cluster	~ 2.0 to 10.0 Mpc	Hundreds of galaxies bound in a single hot gas node.
Typical Cosmic Void	~ 20.0 to 50.0 Mpc	Vast, underdense regions between filaments.

Structure	Approximate Diameter (Mpc)	Notes
Representative Simulation Box	**100.0** + Mpc	The minimum scale required to combat Cosmic Variance.

Initial Conditions

The outcome of a cosmological simulation is critically dependent on its starting point. We cannot simply scatter particles randomly into a box; to accurately model our universe, we must generate a physical snapshot that perfectly captures the primordial density fluctuations that seeded all future cosmic structure.

Fortunately, in the very early universe, these density fluctuations were incredibly tiny. Because the variations in the gravitational field were so weak and smooth, the initial clustering of matter was entirely **linear**. This is a massive advantage for computational cosmology: it means we do not need to waste immense computational resources running an N-body solver from the very beginning of time.

Instead, we can fast-forward through the earliest epochs using a highly accurate analytical framework called the **Zel'dovich Approximation**. Because the physics were still linear, this mathematical approximation can perfectly predict exactly how those tiny primordial ripples displaced matter over millions of years.

However, as matter gradually clumps together, local gravity becomes exponentially stronger and wildly non-linear. Eventually, dark matter streams cross and gas violently collides. At this point, the linear Zel'dovich approximation completely breaks down. That breaking point is our simulation's starting line: we use the Zel'dovich approximation to instantly generate the universe's geometry right up to the edge of the linear regime—typically around a scale factor of $a = 0.02$, roughly 50 million years after the Big Bang. Exactly where this analytical approximation starts to fail, our full numerical solvers take over to compute the chaotic, non-linear birth of galaxies.

The Unperturbed State

To represent the nearly uniform matter distribution of the early universe, we begin by placing particles on a perfect, uniform cubic lattice. For a simulation with N particles in a cubic box of side length L, the initial grid position, $\mathbf{x}_{\text{grid}}$, of a particle is determined by its integer indices (i, j, k).

The position is calculated by determining the number of particles per side, N_s, the spacing between them, d, and then placing each particle at the center of its virtual cubic cell:

The number of particles per side is:

$$N_s = N^{1/3}$$

The spacing between grid points is:

$$d = \frac{L}{N_s}$$

The position vector, $\mathbf{x}_{\text{grid}}$, for the particle at indices (i, j, k) is then given by its components:

$$x = \left(i + \frac{1}{2}\right) d$$

$$y = \left(j + \frac{1}{2}\right) d$$

$$z = \left(k + \frac{1}{2}\right) d$$

Where the indices i, j, and k each run from 0 to $N_s - 1$. The addition of $1/2$ ensures that each particle is placed in the center of its cell, rather than at the corner.

The Zel'dovich Approximation

The real universe was not perfectly uniform. To create the seeds of galaxies and clusters, we must apply small, correlated **perturbations** to our particle lattice. The standard method for this is the **Zel'dovich Approximation**, which generates a smooth displacement field to "nudge" each particle from its perfect grid position.

It is crucial to understand the role of the Zel'dovich Approximation in this context. Although it is an analytical theory that describes the linear growth of structure in the early universe, we do not use it to evolve the particles during our simulation. Instead, we use its time-dependent nature *only once* to generate a single, self-consistent snapshot of the universe at our chosen start time, $t_{initial}$. This snapshot provides both the initial particle positions and their corresponding initial peculiar velocities. From that moment forward, the N-body simulation takes over, calculating the full, non-linear evolution of these particles on its own.

Mathematically, the Zel'dovich Approximation is an application of **first-order Lagrangian perturbation theory (1LPT)**. For higher accuracy more advanced schemes such as **second-order Lagrangian perturbation theory (2LPT)** are often employed, but not covered in this text.

The Growth Factor

Although we only use it to generate a single initial snapshot, the Zel'dovich Approximation is fundamentally a dynamic theory in which the displacement field, $\mathbf{\Psi}$, is not constant. As the universe evolves, the tiny initial overdensities attract more matter, causing the perturbations to grow stronger. In the linear regime, the spatial pattern of the displacement field remains fixed, while its amplitude grows over time. This growth is described by a single function of time, the **linear growth factor**, $D(t)$.

The full, time-dependent displacement field can therefore be written as:

$$\mathbf{\Psi}(\mathbf{x}, t) = D(t)\mathbf{\Psi}_0(\mathbf{x})$$

Here, $\mathbf{\Psi}_0(\mathbf{x})$ is the primordial displacement pattern at some reference time (conventionally, today, where $a = 1$ and $D = 1$), and $D(t)$ scales this entire pattern up or down depending on the cosmic epoch. In a simple Einstein-de Sitter universe model (or a very early ΛCDM, e.g., $a = 0.02$), the growth factor is conveniently proportional to the scale factor, $D(t) \propto a(t)$.

It is crucial to understand that $\mathbf{\Psi}_0(\mathbf{x})$ does not describe the actual highly non-linear universe today. Rather, it is a linearly extrapolated field: a mathematical representation of what the universe would look like today if gravity had remained perfectly linear for 13.8 billion years.

The $D(t)\mathbf{\Psi}_0(\mathbf{x})$ separability is incredibly powerful. It means we only need to compute this complex spatial pattern, $\mathbf{\Psi}_0(\mathbf{x})$, once, anchoring it to present-day observational parameters (like σ_8). To generate the actual state of the universe at our starting epoch, we simply scale this pattern backward in time by multiplying it by the appropriate value of $D(t)$. In cosmological simulations, we define this starting epoch not with a physical time t, but with the initial scale factor, a_{initial} (e.g., $a = 0.02$). Because the fluctuations at this early time are tiny, the linear Zel'dovich approximation becomes physically highly accurate, providing the perfect initial conditions for our N-body particles before gravity drives them into non-linear collapse.

Generating the Displacement Pattern

The process of generating the spatial pattern, $\mathbf{\Psi}_0(\mathbf{x})$, begins in Fourier space with the **power spectrum**, $P(k)$. The power spectrum is the statistical recipe for our universe's initial conditions, specifying the amplitude of density fluctuations at different spatial scales, or wavenumbers (k).

The spectral index, n, is the most important term for defining the *character* of the initial cosmic structure, controlling the balance of power between large-scale (low frequency, k) and small-scale (high frequency, k) fluctuations.

In cosmology, the special "flat" or **scale-invariant** spectrum is defined by a spectral index of $n = 1$. This case, known as the Harrison-Zel'dovich spectrum, represents a universe where the initial fluctuations have the same strength on all physical scales. All real-world spectra are described by how they "tilt" away from this baseline.

- If $n = 1$, we have a **scale-invariant** spectrum. This is the theoretical "white noise" or baseline for cosmology.
- If $n < 1$, we have a **"red-tilted"** spectrum. There is more power in large-scale (low k) fluctuations and less power in small-scale (high k) ones, compared to the scale-invariant case. This results in a universe with large, gentle, rolling waves of density.

- If $n > 1$, we have a **"blue-tilted"** spectrum. There is less power on large scales and more power on small scales, compared to the scale-invariant case.

Observations of the early universe show that our cosmos has a **"red-tilted"** spectrum, with a primordial spectral index n_s very close to 1 (specifically, $n_s \approx 0.96$). This means the initial density ripples were slightly stronger on larger scales than on small ones, providing the seeds for the vast cosmic web we see today.

To generate the displacement pattern, $\mathbf{\Psi}_0(\mathbf{x})$, we use the following steps:

1. **Define the Wavevectors and Physical Scale.** In a 3D Fourier grid, every wave is defined by a **wavevector**, $\mathbf{k} = (k_x, k_y, k_z)$, which points in the direction the wave is traveling. The magnitude of this vector is the **wavenumber**, $k = |\mathbf{k}|$, which represents the wave's spatial frequency. To simulate the real universe, we must anchor our discrete grid to a physical scale. By defining the comoving size of our simulation box in Megaparsecs (L_{box}), we can calculate the exact physical wavenumber for every mode:

$$k_{\text{phys}} = \sqrt{k_x^2 + k_y^2 + k_z^2} \cdot \left(\frac{2\pi}{L_{\text{box}}} \right)$$

   This physical wavenumber tells the simulation exactly what size structure each wave represents, from massive superclusters to small dwarf galaxies.

2. **Generate and Shape the Random Field.** We start by creating a grid of random complex numbers, $\delta(\mathbf{k})$, that satisfies **conjugate symmetry**, $\delta(\mathbf{k}) = \delta^*(-\mathbf{k})$, to ensure the final field in real space is real-valued. Each Fourier mode is then scaled so its amplitude follows the **ΛCDM power spectrum**. While the primordial universe started with a nearly scale-invariant spectrum (k^{n_s}), the presence of intense radiation in the early cosmos suppressed the gravitational collapse of small-scale structures. To capture this physics, we multiply the primordial spectrum by a **Cosmological Transfer Function**, $T(k)$. A standard analytical approximation for this is the **BBKS Transfer Function** (Bardeen, Bond, Kaiser, Szalay, 1986)—explained later. The final shaped power spectrum is evaluated using our physical wavenumbers:

$$P(k_{\text{phys}}) = A \cdot k_{\text{phys}}^{n_s} T(k_{\text{phys}})^2$$

   Here, A is a master normalization constant that scales the overall strength of the fluctuations. Each random mode is scaled by the square root of this power spectrum: $\delta_\rho(\mathbf{k}) = \delta(\mathbf{k})\sqrt{P(k_{\text{phys}})}$.

3. **Compute the displacement field.** The random field $\delta_\rho(\mathbf{k})$ we just generated is a scalar map of density (where the mass is). However, our goal is the displacement field $\mathbf{\Psi}_0(\mathbf{x})$, which is a vector map telling particles which way to move. To bridge this gap, we must use the physics of gravity. In the Zel'dovich approximation, particles are displaced by falling down the gradient of the gravitational potential created by the density fluctuations. First, we use Poisson's equation to convert the density field into a gravitational potential $\hat{\Phi}(\mathbf{k})$, which requires an inverse Laplacian operation (dividing by $-k^2$). Second, we take the gradient of this potential to find the displacement vector, which in Fourier space corresponds to multiplying by $i\mathbf{k}$. Because this derivative is taking place purely on our discrete grid, we drop the physical Megaparsec scaling and strictly use the dimensionless internal grid-unit wavevectors ($\mathbf{k}_{\text{grid}}$):

$$\hat{\mathbf{\Psi}}_0(\mathbf{k}) \propto i\mathbf{k}_{\text{grid}} \frac{\delta_\rho(\mathbf{k})}{k_{\text{grid}}^2}$$

4. **Transform back to real space.** Finally, we apply the inverse Fourier transform to recover the displacement pattern in real space:

$$\mathbf{\Psi}_0(\mathbf{x}) = \mathcal{F}^{-1}\{\hat{\mathbf{\Psi}}_0(\mathbf{k})\}$$

Normalizing the Power Spectrum (σ_8) In the previous step, we left the overall amplitude multiplier, A, undefined. To anchor our initial conditions to observational reality, this amplitude cannot be arbitrary. In cosmology, it is pinned to a standard measured value known as σ_8 (Sigma-8).

σ_8 represents the root-mean-square (RMS) variance of mass density fluctuations within a sphere of radius 8 Mpc/h in the present-day universe. If you were to drop spheres of this size randomly throughout the cosmos, the mass inside them would vary depending on whether they landed in an empty void or a dense supercluster. σ_8 quantifies this variance. Current observations (such as those from the Planck satellite) show that for our universe, $\sigma_8 \approx 0.81$.

To enforce this in our simulation, we must normalize our theoretical power spectrum so that its mathematical variance at $R = 8$ Mpc/h exactly equals σ_8^2. The variance σ_R^2 of a field smoothed over a physical scale R is

found by integrating the power spectrum multiplied by a "window function," $\tilde{W}(kR)$, in Fourier space:

$$\sigma_R^2 = \frac{1}{2\pi^2}\int_0^\infty P_{\text{unnorm}}(k)\tilde{W}^2(kR)k^2 dk$$

For a spherical volume, the appropriate filter is the **spherical top-hat window function**, whose Fourier transform is:

$$\tilde{W}(kR) = \frac{3\left(\sin(kR) - kR\cos(kR)\right)}{(kR)^3}$$

By numerically integrating our unnormalized BBKS power spectrum (where $A = 1$) using this window function at $R = 8$, we calculate the unnormalized variance. The master normalization constant, A, is then simply the ratio of the target observational variance to this theoretical variance:

$$A = \frac{\sigma_8^2}{\sigma_{R=8,\text{unnorm}}^2}$$

Applying this constant A ensures that the resulting displacement field possesses the exact statistical "clumpiness" observed in the real universe.

The Physics of the Transfer Function: BBKS If the universe contained only dark matter, the primordial power spectrum ($P(k) \propto k^{n_s}$) would remain nearly a straight line across all scales. However, the early universe was a chaotic, boiling soup dominated by intense radiation. This radiation fundamentally altered how structures of different sizes were allowed to grow, a process we capture mathematically using the **Cosmological Transfer Function**, $T(k)$.

The shape of $T(k)$ is driven by a cosmic race between gravity and the expansion of space, dictated by the **horizon** (the maximum distance light, and therefore any causal physical interaction, could have traveled since the Big Bang).

1. **Large Scales (Small k):** These density fluctuations were physically larger than the cosmic horizon in the early universe. Because gravity cannot act faster than light, these regions were effectively "frozen" outside the horizon. By the time the horizon grew large enough to encompass them, the universe had already cooled and transitioned into the **matter-dominated era**. In this calm era, gravity easily took over, and these large scales began to collapse normally. Therefore, for small k, the transfer function does nothing: $T \approx 1$.
2. **Small Scales (Large k):** These tiny fluctuations were small enough to enter the horizon very early on, right in the middle of the **radiation-dominated era**. During this epoch, the pressure of the radiation drove a tremendously fast expansion of space. This expansion was so violent that it outpaced the gravitational pull of the dark matter. The growth of these small-scale density ripples was completely stalled—a phenomenon known as the **Mészáros effect**. They could only start growing later, once the radiation cleared. Because they lost millions of years of growth time, their final amplitude is heavily suppressed.

To calculate the exact shape of this suppression, cosmologists must solve complex, coupled differential equations tracking dark matter, baryons, photons, and neutrinos. In 1986, Bardeen, Bond, Kaiser, and Szalay (BBKS) published a masterful analytical fitting formula that approximates the result of these complex calculations.

The BBKS transfer function depends on a scaled wavenumber, q, which adjusts the physical wavenumber k (measured in Mpc^{-1}) based on the density of the universe. For a standard model, it is approximated as:

$$q = \frac{k}{\Omega_m h^2 \exp(-\Omega_b - \sqrt{2h}\Omega_b/\Omega_m)}$$

(Note: In purely dark-matter-dominated, simplified models, the denominator is often reduced to simply the "shape parameter" $\Gamma \approx \Omega_m h$.)

The BBKS formula itself is:

$$T(q) = \frac{\ln(1+2.34q)}{2.34q}\left[1 + 3.89q + (16.1q)^2 + (5.46q)^3 + (6.71q)^4\right]^{-0.25}$$

Mathematically, this equation perfectly captures the physics of the early universe:

- As $q \to 0$ (huge cosmic scales), the function approaches 1, preserving the primordial k^{n_s} shape.

- As $q \to \infty$ (tiny cosmic scales), the function falls off proportionally to q^{-2}.

Because the final power spectrum is proportional to $T(k)^2$, this causes the small-scale power to drop off by a massive factor of k^{-4}. This creates a distinct "turnover" peak in the total power spectrum—a critical signature of the transition from the radiation era to the matter era, and the defining mathematical curve that dictates the sizes of galaxies in our simulation.

Applying the Displacements and Velocities

With the spatial pattern $\mathbf{\Psi}_0(\mathbf{x})$ calculated and perfectly normalized to the present-day universe ($a = 1$), we can now set the initial state of our simulation at its starting time. Because the normalization is baked into the field, applying it to the early universe relies entirely on cosmological scaling.

The final initial position of each particle is its grid position plus the displacement field, scaled back in time by the initial linear growth factor, $D(t)$. In the very early universe (e.g., $a = 0.02$), matter overwhelmingly dominates over Dark Energy. Because of this, we can safely approximate that the early growth factor scales directly with the expansion of space, $D(a) \approx a$.

$$\mathbf{x}_{\text{final}} = \mathbf{x}_{\text{grid}} + a_{\text{initial}} \mathbf{\Psi}_0(\mathbf{x}_{\text{grid}})$$

However, calculating the initial "peculiar" velocity (a particle's motion on top of the Hubble flow) requires more care. The velocity is the time derivative of the comoving displacement, meaning it depends on the *rate of change* of the growth factor:

$$\mathbf{v}_{\text{pec}} = \left.\frac{dD(t)}{dt}\right|_{\text{initial}} \mathbf{\Psi}_0(\mathbf{x}_{\text{grid}})$$

In a pure Einstein-de Sitter universe, this derivative simplifies neatly to $\frac{dD}{dt} = H(t)D(t)$. But in a full ΛCDM universe, we must account for the fact that Dark Energy is actively suppressing the rate at which these structures grow. To express this physically, cosmologists define the **Logarithmic Growth Rate**, f:

$$f = \frac{d\ln D}{d\ln a}$$

A highly accurate standard approximation for this rate (Peebles, 1980) depends on the matter density at that specific epoch:

$$f \approx \Omega_m(a)^{0.55}$$

By substituting this growth rate into our derivative, we arrive at the generalized, fully cosmological equation for the initial peculiar velocities. It is proportional to the displacement field itself, scaled by the Hubble parameter, the scale factor, and the critical suppression factor f:

$$\mathbf{v}_{\text{pec}} = H_{\text{initial}} \cdot a_{\text{initial}} \cdot f \cdot \mathbf{\Psi}_0(\mathbf{x}_{\text{grid}})$$

This method produces a self-consistent set of initial conditions for both position and velocity, perfectly tailored to the chosen cosmology. The particle motions are correlated over large distances, forming the beginnings of the filaments and voids that will later evolve into galaxies and clusters.

Key Literature & Further Reading
Sirko, E. (2005). *Initial Conditions to Cosmological N-Body Simulations, or Translations from the Power Spectrum to Real Space.* arXiv:astro-ph/0503106. Available at https://arxiv.org/abs/astro-ph/0503106

Validation and Accuracy

Conservation of Energy and Momentum

A physically meaningful simulation must obey the same fundamental laws as the universe it models. For a closed system governed by gravity, the two most powerful checks are therefore the laws of conservation of energy and momentum. If a simulation violates these "golden rules", it is a clear sign of a fundamental error in its implementation or underlying model.

It is crucial to understand that the rules of **conservation of energy and momentum** are only valid for a closed, non-expanding system. The total energy of a system of particles is *not* a conserved quantity in an expanding universe. The expansion of space itself does work on the system. The peculiar velocities of particles

decrease due to Hubble drag, and the potential energy changes as the physical distances between all particles grow. Therefore, checking for energy conservation during a cosmological run is not a valid test; the energy is expected to change over time.

To use conservation as a test of a simulation's accuracy, we must first disable the cosmic expansion. This is done by setting the scale factor to a constant value, $a(t) = 1$, for the entire run. In this "static box" mode, the universe does not expand, and the Hubble drag term is zero. The simulation becomes a pure gravitational N-body problem.

The standard practice is to first validate the code in a non-expanding box to confirm these conservation laws hold to a high degree. Once the core engine is verified, we can then enable the cosmic expansion, confident that any subsequent physical behavior is a result of the cosmology, not a bug in the integrator.

Conservation of Momentum

In a closed system with no external forces, the total momentum must remain constant. The conservation of momentum is a direct consequence of Newton's third law: for every force $\mathbf{F}_{ij}$ that particle j exerts on particle i, there is an equal and opposite force $\mathbf{F}_{ji} = -\mathbf{F}_{ij}$. When we sum the forces over the entire system, all these internal forces perfectly cancel out. If the code correctly implements this symmetry, total momentum will be conserved.

The total momentum $\mathbf{P} = (P_x, P_y, P_z)$ can be computed as:

$$P_x = \sum_i m_i v_{ix}$$

$$P_y = \sum_i m_i v_{iy}$$

$$P_z = \sum_i m_i v_{iz}$$

Where m_i is the mass of particle i, and v_{ix}, v_{iy} and v_{iz} are its velocity components. The values of P_x, P_y and P_z should remain unchanged along the simulation to within machine precision. Any systematic drift indicates a bug in the force calculation.

In a closed system with no external torques, the total **angular momentum** must also be conserved. For a system of particles, the total angular momentum is the vector sum of each particle's individual angular momentum, $\mathbf{L} = \mathbf{r} \times \mathbf{p}$.

We can calculate the total angular momentum vector $\mathbf{L} = (L_x, L_y, L_z)$ using the formula:

$$L_x = \sum_i m_i (y_i v_{iz} - z_i v_{iy})$$

$$L_y = \sum_i m_i (z_i v_{ix} - x_i v_{iz})$$

$$L_z = \sum_i m_i (x_i v_{iy} - y_i v_{ix})$$

Just like with linear momentum, the values of L_x, L_y, and L_z should each remain unchanged. Any systematic drift in any component signals a subtle bug in the geometric implementation of your force.

Conservation of Energy

For a conservative system like gravity, the total energy—the sum of the **Kinetic Energy (KE)** from motion and the **Potential Energy (PE)** from gravitational attraction—must remain constant.

This is a much more sensitive and comprehensive test than momentum conservation. It validates the entire simulation loop, especially the accuracy of the **time integrator**. While momentum checks the symmetry of the forces at a single instant, energy conservation checks how well the simulation evolves the system from one state to the next.

Because the simulation moves in discrete time steps, we don't expect the energy to be conserved perfectly. Instead, the *behavior* of the error tells us if the integrator is working correctly:

- **A good (symplectic) integrator** will produce an energy error that **oscillates** around the initial value. The energy will wobble, but it will not show a long-term trend of increasing or decreasing.
- **A bad or inconsistent integrator** will cause the energy to **drift** steadily over time, indicating a systematic error that is continuously adding or removing energy from the system.

To calculate the total energy, $E(0) = KE + PE$ we just add the kinetic and potential energies as described below.

- **Kinetic Energy (KE):** The total kinetic energy is the sum of the kinetic energy of every particle in the system.

$$KE = \sum_{i=1}^{N} \frac{1}{2} m_i v_i^2$$

  Where m_i is the mass of particle i and v_i is its speed ($v_i^2 = v_{ix}^2 + v_{iy}^2 + v_{iz}^2$).

- **Potential Energy (PE):** The total potential energy is the sum of the potential energy for every *unique pair* of particles. The most meaningful way to measure the simulation's accuracy is to check how well it conserves the total energy of the **ideal, unsoftened system**.

$$PE = \sum_{i=1}^{N} \sum_{j>i}^{N} \frac{-G m_i m_j}{r_{ij}}$$

  Where r_{ij} is the distance between particles i and j.

We can periodically recalculate this total energy, $E(t)$, as the simulation runs and monitor the relative error over time: $\frac{E(t)-E(0)}{E(0)}$.

A small, bounded oscillation in this value is the signature of a healthy, stable simulation. A consistent drift, no matter how small, points to a deeper issue that needs to be fixed.

The Two-Body Problem and Kepler's Laws

The conservation laws of energy and momentum are powerful checks on the overall stability of our simulation, but they don't tell us if the individual trajectories of our particles are physically correct. For that, we need a "ground truth"—a simple problem with a known, exact mathematical solution that we can compare our simulation against.

In celestial mechanics, the perfect "unit test" is the **two-body problem**. It is one of the very few gravitational problems that can be solved perfectly with pen and paper, and its solution is described by **Kepler's Laws of Planetary Motion**. If a simulation can't reproduce this fundamental result, it can't be trusted with more complex systems.

The setup requires to simplify the simulation to just two bodies:

1. **The "Star":** Create one particle with a very large mass and place it near the center of the simulation box. Give it zero initial velocity to keep it mostly stationary.
2. **The "Planet":** Create a second particle with a much smaller mass and place it some distance away from the star, for example, along the x-axis. Give the planet an initial velocity purely in the y-direction. The magnitude of this velocity is crucial; a specific value will produce a circular orbit, while slightly different values will produce stable elliptical orbits.

After running the simulation, we can check the planet's trajectory against Kepler's predictions:

1. Is the Orbit a Closed Ellipse? (Kepler's First Law) The most important check. The planet should trace a stable, closed ellipse with the star at one of the foci. Common failure modes are:

- **Energy Drift:** The orbit spirals inwards or outwards, indicating a non-symplectic integrator or a bug.
- **Unphysical Precession:** The ellipse itself rotates over time. A large, rapid precession is a sign of numerical inaccuracy. A stable, non-precessing ellipse is a sign of a healthy integrator.

2. Does it Speed Up and Slow Down Correctly? (Kepler's Second Law) This law states that a planet sweeps out equal areas in equal times, which is a consequence of angular momentum conservation. This means the planet must move **fastest** when it is closest to the star (perihelion) and **slowest** when it is farthest away (aphelion).

3. Does it Obey the Law of Periods? (Kepler's Third Law) The law states that the square of the orbital period (P) is proportional to the cube of the orbit's semi-major axis (a), or $P^2 \propto a^3$. The time it takes the simulated planet to complete one full orbit and the size of its orbit must satisfy this mathematical relationship.

Passing the two-body test is a critical milestone. It builds confidence that the implementations of the gravitational force and the time integrator are fundamentally sound, and it's a prerequisite for tackling the chaotic dance of a true N-body system.

Sources of Error

A perfect simulation is impossible. The goal is not to eliminate all errors but to understand where they come from, control them, and ensure they are small enough that the results are physically meaningful. In a P^3M simulation, the errors arise from the three fundamental approximations we make to turn the smooth, continuous problem of gravity into a discrete problem a computer can solve.

Time Discretization Error

Physics happens continuously, but a simulation must leap forward in discrete chunks of time, Δt. The time integrator works by assuming that the acceleration changes in a simple, predictable way during that small leap.

However, the true acceleration, especially during a close encounter, can be more complex. The difference between the integrator's assumed path and the true physical path is called **truncation error**. This error scales with the square of the time step ($O(\Delta t^2)$) and is a primary contributor to the oscillations in the total energy.

Softening Bias

To prevent infinite forces when particles get too close, we modify the force law with a softening parameter, ϵ.

$$F = \frac{Gm_1m_2}{r^2 + \epsilon^2}$$

This is not a numerical error but a **physical modeling error**. We are knowingly simulating a slightly different, "softer" version of gravity. This error is most significant at very short distances ($r \lesssim \epsilon$) and prevents the formation of very "hard," dense structures. We accept this trade-off because it grants us numerical stability, but the simulation becomes blind to any physics occurring at scales smaller than the softening length.

Spatial Discretization Error

The Particle-Mesh method gains its speed by calculating long-range forces on a grid. This introduces several errors related to the grid's finite resolution.

- **Aliasing:** The grid can only represent waves with a wavelength larger than about two cell sizes. When two particles get closer than this, their sharp, high-frequency density spike is misinterpreted by the grid as a smooth, low-frequency wave. This is **aliasing**, and it is the primary source of inaccuracy in the PM force, making it "blurry" and even repulsive at short distances.
- **Interpolation Error:** The schemes used to assign mass to the grid and interpolate forces back (like NGP or CIC) are approximations that contribute to the total error.
- **Finite Difference Error:** The force on the grid is calculated using a finite difference approximation of the gradient. This is not a perfect derivative and adds a small amount of error.

Ultimately, running a successful simulation implies balancing these interconnected errors. A smaller softening length demands a smaller time step. A finer grid reduces aliasing but increases computational cost. Understanding these trade-offs is the key to producing results that are both stable and physically reliable.

Key Literature & Further Reading
Dehnen, W., & Read, J. I. (2011). *N-body simulations of gravitational dynamics.* arXiv:1105.1082. Available at https://arxiv.org/abs/1105.1082.

Hydrodynamics

A simulation that focuses on the evolution of collisionless N-body particles is an excellent model for the dark matter that forms the universe's invisible scaffolding. However, to simulate the formation of the luminous structures—stars, galaxies, and galaxy clusters—it must include the physics of **baryonic matter**. In cosmology, "baryons" is the term for all normal matter, which primarily exists as a vast, diffuse **gas**.

Unlike dark matter, gas is not collisionless. Its particles (atoms and ions) interact with each other, giving rise to familiar fluid properties like **pressure** and **temperature**.

However, it is natural to wonder how a medium so incredibly empty—often containing just a single atom in a volume the size of a small room—can behave like a fluid and push back against gravity. The answer lies in the sheer scale of the universe and the physics of plasmas.

Out in the deep cosmic voids, the primordial gas is very cold and diffuse. However, because cosmological structures are millions of light-years across, the distance an atom travels before colliding with another is still microscopic compared to the size of the system. On these vast scales, even a near-vacuum experiences enough microscopic collisions to possess macroscopic fluid properties like pressure and density.

The physics changes dramatically when this cold gas falls into the immense gravity of a dark matter halo. As the gas crashes into the halo at hypersonic speeds, shockwaves heat it to millions of degrees, turning it into a highly energetic plasma. In this state, the charged ions and electrons do not need to physically collide like billiard balls to interact. Instead, they repel each other across vast distances via electromagnetic forces and are threaded together by large-scale magnetic fields. These long-range interactions ensure that even inside the hottest, most violent galaxy clusters, the diffuse matter continues to act as a cohesive fluid.

To mathematically model this continuous behavior across both the cold voids and the hot halos, a simulation must solve the laws of **hydrodynamics**.

The Euler Equations

For a simple, non-viscous gas (a good approximation for most cosmic fluids), its motion is governed by a set of three conservation laws known as the **Euler Equations**. These equations describe how the density, momentum, and energy of the gas change over time.

In a simulation, these equations are solved on the same grid used for the Particle-Mesh gravity solver. This is known as an **Eulerian** approach, where the properties of the fluid are tracked as it flows through our fixed grid cells.

1. Conservation of Mass

This law states that the change in the mass density (ρ) in a given cell is equal to the net flow of mass into or out of it. If more gas flows in than out, the density increases.

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{v}) = 0$$

Here, $\mathbf{v}$ is the velocity of the gas, and the term $\nabla \cdot (\rho \mathbf{v})$ represents the divergence, or the "outflow," of the mass flux. The divergence of this vector is simply the sum of the partial spatial derivatives of each component along its respective axis:

$$\nabla \cdot (\rho \mathbf{v}) = \frac{\partial (\rho v_x)}{\partial x} + \frac{\partial (\rho v_y)}{\partial y} + \frac{\partial (\rho v_z)}{\partial z}$$

2. Conservation of Momentum

This law states that the change in the gas's momentum ($\rho \mathbf{v}$) is caused by the forces acting on it. In a cosmological simulation, there are two crucial forces: pressure and gravity.

$$\frac{\partial (\rho \mathbf{v})}{\partial t} + \nabla \cdot (\rho \mathbf{v} \otimes \mathbf{v}) = -\nabla P + \rho \mathbf{g}$$

To understand this equation, we can break it down into the flow of momentum (the left side) and the forces causing it to change (the right side).

The left-hand side describes the movement of the fluid:

- $\frac{\partial (\rho \mathbf{v})}{\partial t}$**:** The local rate of change of momentum density over time.
- $\nabla \cdot (\rho \mathbf{v} \otimes \mathbf{v})$ **(Momentum Advection):** This term describes how the bulk flow of the fluid carries its own momentum across space. The outer product ($\otimes$) creates a tensor (a 3x3 matrix) that tracks how each

directional component of momentum (e.g., y-momentum) is transported along every spatial axis (e.g., flowing along the x-axis).

$$\rho\mathbf{v}\otimes\mathbf{v} = \begin{bmatrix} \rho v_x v_x & \rho v_x v_y & \rho v_x v_z \\ \rho v_y v_x & \rho v_y v_y & \rho v_y v_z \\ \rho v_z v_x & \rho v_z v_y & \rho v_z v_z \end{bmatrix}$$

$$\nabla\cdot(\rho\mathbf{v}\otimes\mathbf{v}) = \begin{bmatrix} \frac{\partial}{\partial x}(\rho v_x v_x) + \frac{\partial}{\partial y}(\rho v_x v_y) + \frac{\partial}{\partial z}(\rho v_x v_z) \\ \frac{\partial}{\partial x}(\rho v_y v_x) + \frac{\partial}{\partial y}(\rho v_y v_y) + \frac{\partial}{\partial z}(\rho v_y v_z) \\ \frac{\partial}{\partial x}(\rho v_z v_x) + \frac{\partial}{\partial y}(\rho v_z v_y) + \frac{\partial}{\partial z}(\rho v_z v_z) \end{bmatrix}$$

The right-hand side represents the physical forces acting on the fluid:

- $-\nabla P$ **(The Pressure Gradient Force):** This is the key new piece of physics. It describes the force that causes gas to flow from regions of high pressure to regions of low pressure. This is the force that allows the gas to resist gravitational collapse.

$$-\nabla P = \begin{bmatrix} -\frac{\partial P}{\partial x} \\ -\frac{\partial P}{\partial y} \\ -\frac{\partial P}{\partial z} \end{bmatrix}$$

- $\rho\mathbf{g}$ **(The Gravitational Force):** This is the familiar force of gravity. The gravitational acceleration, $\mathbf{g}$, is calculated from the density of **all** matter (both dark matter and gas) using the existing Particle-Mesh solver. This term is the link that couples the gas to the underlying cosmic web.

3. Conservation of Energy and the Equation of State

The full energy conservation equation is complex, but for a simple simulation, we can track the **internal energy** of the gas, e, which is a measure of its temperature. The pressure, P, is not an independent variable but is related to the density and internal energy by an **equation of state**. For a simple, ideal gas, this equation is:

$$P = (\gamma - 1)\rho e$$

Here, γ is the adiabatic index, a constant which is typically 5/3 for a monatomic gas like the hydrogen and helium that fill the cosmos. It describes how much the pressure of a gas responds to a change in volume during an adibatic process—the gas is compressed or expands so quickly that it doesn't have time to exchange heat with its surroundings. A higher γ means the pressure rises more sharply for the same amount of compression.

Hybrid Simulation

In a **hybrid code**, the dark matter is treated as a collection of Lagrangian N-body particles that follow trajectories, while the gas is treated as a continuous Eulerian fluid, whose properties are tracked on a grid. The two components are linked together by the force of gravity, which is sourced by their combined density. This hybrid approach is the foundation of all modern cosmological simulations that aim to model the formation of the visible universe.

An Operator-Split Hydro-Solver

To simulate the gaseous (baryonic) component, we can adopt an **Eulerian** approach, solving the equations of hydrodynamics on the same fixed grid used by the Particle-Mesh gravity solver.

The full set of cosmological hydrodynamic equations is complex, as it couples the conservation laws of fluid dynamics with the source terms from gravity in an expanding universe. The equation for the vector of conserved quantities, $\mathbf{U} = [\rho, \rho\mathbf{v}, E]$, can be written as:

$$\frac{\partial\mathbf{U}}{\partial t} + \nabla\cdot\mathbf{F}(\mathbf{U}) = \mathbf{S}(\mathbf{U})$$

Where:

- $\mathbf{U}$ is the vector of conserved state variables (density, momentum density, energy density).
- $\nabla\cdot\mathbf{F}(\mathbf{U})$ is the "flux" term, which describes how quantities move due to pressure and advection (the fluid flowing).

- $\mathbf{S}(\mathbf{U})$ is the "source" term, which describes changes due to external forces, like gravity ($\rho\mathbf{g}$).

While standard fluid dynamics relies only on those three fundamental conservation laws, simulating the universe requires a mathematical safety net. In cosmological flows, gas often moves at extreme, hypersonic speeds. In these conditions, a cell's kinetic energy becomes so massive that trying to numerically extract the tiny fraction of internal thermal energy from the total energy (E) leads to catastrophic floating-point errors.

To solve this, our code explicitly tracks a fourth variable: **internal energy density (ie)**. Therefore, for the remainder of this text, we will expand our state vector to a four-element array:

$$\mathbf{U} = [\rho, \rho\mathbf{v}, E, ie]$$

The precise mechanics of this issue and how this fourth variable is evolved alongside the others is known as the **Dual Energy Formalism**, which will be explored in detail in a later section.

This is the perfect place to formally expand the equation to four rows. It solidifies the "pivot" we just made and gives the reader the complete, unified mathematical picture of your code's exact architecture.

When we add the internal energy (ie) row, its flux is simply the advection of that thermal energy. However, its "source" term requires a bit of care: gravity doesn't directly heat up the gas (it changes the bulk kinetic energy), but the gas *does* heat up or cool down as it compresses or expands. Therefore, the source term for row 4 is the internal PdV work.

To reflect our 4-element state vector $\mathbf{U}$, the expanded system of equations looks like this:

$$\frac{\partial}{\partial t}\begin{bmatrix} \rho \\ \rho\mathbf{v} \\ E \\ ie \end{bmatrix} + \nabla \cdot \begin{bmatrix} \rho\mathbf{v} \\ \rho\mathbf{v} \otimes \mathbf{v} + P\mathbf{I} \\ (E+P)\mathbf{v} \\ ie\mathbf{v} \end{bmatrix} = \begin{bmatrix} 0 \\ \rho\mathbf{g} \\ \rho\mathbf{v} \cdot \mathbf{g} \\ -P(\nabla \cdot \mathbf{v}) \end{bmatrix}$$

Here is the breakdown of exactly what each row represents:

- **Row 1 (Conservation of Mass):**
  - **Flux:** $\rho\mathbf{v}$ is the advection of mass.
  - **Source:** 0, because the universe does not spontaneously create or destroy mass.
- **Row 2 (Conservation of Momentum):**
  - **Flux:** $\rho\mathbf{v} \otimes \mathbf{v} + P\mathbf{I}$. This combines the momentum advection tensor ($\rho\mathbf{v} \otimes \mathbf{v}$) with the thermal pressure (P). The $\mathbf{I}$ is the identity matrix, which is just a mathematical way of ensuring the pressure only acts perpendicular to the cell faces (which exactly matches the $-\nabla P$ term when the divergence operator is applied).
  - **Source:** $\rho\mathbf{g}$, the external acceleration due to gravity pulling on the gas.
- **Row 3 (Conservation of Total Energy):**
  - **Flux:** $(E+P)\mathbf{v}$. This is the total energy sliding along with the gas ($E\mathbf{v}$), plus the mechanical PdV work being done by the fluid as it compresses and expands against its neighbors ($P\mathbf{v}$).
  - **Source:** $\rho\mathbf{v} \cdot \mathbf{g}$. This is the mechanical work done *by gravity*. As gas falls into a dark matter halo (velocity aligns with gravity), gravity does work on the gas, adding to its total energy.
- **Row 4 (Internal Energy / Dual Energy Formalism):**
  - **Flux:** $ie\mathbf{v}$. This is the pure advection of thermal energy (hot gas flowing from one place to another).
  - **Source:** $-P(\nabla \cdot \mathbf{v})$. This is the internal thermodynamic source term. Gravity does not directly heat the gas; instead, the gas heats up or cools down purely based on how its volume changes. As the fluid converges and compresses ($\nabla \cdot \mathbf{v} < 0$), this term becomes positive, acting as a source that heats the gas.

Solving this equation all at once is difficult. A common and effective technique called **operator splitting** breaks the problem into simpler, sequential steps.

Here is the step-by-step process to advance the gas grid from time t to $t + \Delta t$ following the **Kick-Drift-Kick (KDK)** structure.

Step 1: Convert to Primitive Variables

(A quick note on notation: In the generalized Euler equations above, we used $\mathbf{S}(\mathbf{U})$ *to represent the full "Source Vector." However, from this point forward, we will adopt the standard simulation nomenclature where the vector* $\mathbf{S}$ *(and its components* S_x, S_y, S_z*) refers specifically to the* ***momentum density****,* $\rho\mathbf{v}$*.)*

The solver begins with the grid of **conserved variables** (ρ, $S_x = \rho v_x$, $S_y = \rho v_y$, E) from the previous step. To calculate fluxes and forces, it first computes the **primitive variables** (velocity $\mathbf{v}$ and pressure P):

$$v_x = \frac{S_x}{\rho} \quad , \quad v_y = \frac{S_y}{\rho}$$

$$P = (\gamma - 1)\left(E - \frac{1}{2}\rho|\mathbf{v}|^2\right)$$

Step 2: Gravity Half-Step (Kick)

First, the external forces from gravity are applied for half a time step, $\Delta t/2$. The gravitational acceleration field, $\mathbf{g}$, can be provided by the Particle-Mesh solver. The acceleration field must be derived from the **total matter density**—the sum of the dark matter density (from the N-body particles) and the gas density (from the hydro grid). This step updates the momentum and total energy of the gas:

$$\mathbf{S}(t + \tfrac{1}{2}\Delta t) = \mathbf{S}(t) + (\rho\mathbf{g})\frac{\Delta t}{2}$$

$$E(t + \tfrac{1}{2}\Delta t) = E(t) + (\mathbf{v} \cdot \rho\mathbf{g})\frac{\Delta t}{2}$$

The energy is updated by the **power density** (Force Density · Velocity, or $\mathbf{v} \cdot \rho\mathbf{g}$), which is the rate at which the gravitational field does work on the gas.

Step 3: Hydrodynamic Full-Step (Drift)

With the gravitational source terms applied, we can solve the pure hydrodynamic equations (the "flux" part) for a full time step, Δt. To calculate how much gas flows from one cell to the next, we must first estimate the fluid's properties exactly at the boundary between them—a process known as **spatial reconstruction**.

If we use a high-accuracy, second-order spatial reconstruction scheme (like MUSCL, detailed in the next section), taking a simple first-order "forward Euler" step in time would introduce instabilities and dampen the sharp shocks that we need to resolve. Instead, we must match the spatial accuracy with **second-order time integration**. A highly robust method for finite-volume hydrodynamics is the **Strong Stability Preserving Runge-Kutta (SSP-RK2)** scheme.

Because operator splitting temporarily isolates the fluid's motion (the "Drift" stage) into a purely hydrodynamic problem, we are free to solve it using its own specialized mathematical engine. Therefore, nested inside the global KDK loop, this step utilizes the SSP-RK2 scheme. KDK orchestrates the overall physics, while SSP-RK2 acts as the high-precision sub-engine that actually pushes the gas across the grid.

In multi-stage Runge-Kutta methods, it is standard to denote the fluid state at the current time t using the step index n (written as $\mathbf{U}^n$), and the final state at $t + \Delta t$ as $\mathbf{U}^{n+1}$. The SSP-RK2 method achieves second-order accuracy by breaking the fluid advection into two intermediate mathematical stages (denoted by parenthetical superscripts) and averaging the results:

1. The Predictor Stage: The solver takes a full Euler step using the current fluid state ($\mathbf{U}^n$) to calculate the fluxes. This generates an intermediate, predicted state:

$$\mathbf{U}^{(1)} = \mathbf{U}^n + \Delta t \cdot \mathbf{L}(\mathbf{U}^n)$$

Where $\mathbf{L}(\mathbf{U})$ acts as the **discrete spatial operator**. Earlier in the chapter, the fluid's motion was described by the continuous calculus term $\nabla \cdot \mathbf{F}(\mathbf{U})$. Because our simulation cannot perform infinite calculus, $\mathbf{L}(\mathbf{U})$ serves as the numerical approximation of that physics on a discrete grid. Mathematically, it is defined as:

$$\mathbf{L}(\mathbf{U}) \approx -\nabla \cdot \mathbf{F}(\mathbf{U}) + \mathbf{S}_{\text{internal}}$$

Here, the negative sign appears because divergence ($\nabla \cdot \mathbf{F}$) measures the net **outflow**, whereas $\mathbf{L}(\mathbf{U})$ calculates the net **influx** across the cell boundaries (which is evaluated using the HLLC Riemann solver). The $\mathbf{S}_{\text{internal}}$ term accounts for internal thermodynamics, specifically the PdV work required by the Dual Energy Formalism. Noticeably absent is the external gravity source term ($\rho\mathbf{g}$); because we are using an operator-split Kick-Drift-Kick architecture, gravity is handled entirely separately during the half-steps.

To see exactly how this discrete spatial operator updates the fluid, we can expand the equation $\mathbf{U}^{(1)} = \mathbf{U}^n + \Delta t \cdot \mathbf{L}(\mathbf{U}^n)$ into its full column-vector form.

By mapping the state vector $\mathbf{U}$ to our four tracked variables (mass, momentum, total energy, and internal energy) and expanding $\mathbf{L}(\mathbf{U})$ into its flux and source components, the predictor stage looks like this:

$$\begin{bmatrix} \rho^{(1)} \\ (\rho\mathbf{v})^{(1)} \\ E^{(1)} \\ ie^{(1)} \end{bmatrix} = \begin{bmatrix} \rho^n \\ (\rho\mathbf{v})^n \\ E^n \\ ie^n \end{bmatrix} + \Delta t \cdot \begin{bmatrix} -\nabla \cdot (\rho\mathbf{v})^n \\ -\nabla \cdot (\rho\mathbf{v} \otimes \mathbf{v} + P\mathbf{I})^n \\ -\nabla \cdot ((E+P)\mathbf{v})^n \\ -\nabla \cdot (ie\mathbf{v})^n - P^n(\nabla \cdot \mathbf{v}^n) \end{bmatrix}$$

Looking at the $\mathbf{L}(\mathbf{U}^n)$ operator (the rightmost matrix), you can see exactly how the physics are applied:

- **Rows 1 through 3:** These are the pure flux divergences ($-\nabla \cdot \mathbf{F}$). The Riemann solver calculates the flow of mass, momentum, and total energy across the cell boundaries.
- **Row 4 (Internal Energy):** This row explicitly shows the Dual Energy Formalism at work. It includes both the advection of thermal energy ($-\nabla \cdot (ie\mathbf{v})$) *and* the internal thermodynamics source term ($-P\nabla \cdot \mathbf{v}$), which acts to heat or cool the gas as it compresses or expands during the time step.

2. The Corrector Stage: The solver calculates an entirely new set of fluxes based on the *intermediate* state ($\mathbf{U}^{(1)}$). It takes another full Euler step from this intermediate state to generate a second state:

$$\mathbf{U}^{(2)} = \mathbf{U}^{(1)} + \Delta t \cdot \mathbf{L}(\mathbf{U}^{(1)})$$

3. Final Averaging: Finally, the true state at the next time step ($\mathbf{U}^{n+1}$) is found by averaging the original state and the twice-stepped state:

$$\mathbf{U}^{n+1} = \frac{1}{2}\mathbf{U}^n + \frac{1}{2}\mathbf{U}^{(2)}$$

This staggered, averaging approach allows the gas to flow across the grid with true second-order accuracy in both space and time. While calculating the fluxes twice per cycle essentially doubles the computational cost of the hydrodynamics, the dramatic improvement in shock resolution and overall stability makes it a strict requirement for modern cosmological simulations.

Step 4: Second Gravity Half-Step (Kick)

Finally, the gravitational source terms are applied again for the second half of the time step, $\Delta t/2$, using the updated values from the "Drift" step:

$$\mathbf{S}(t + \Delta t) = \mathbf{S}(t + \tfrac{1}{2}\Delta t) + (\rho\mathbf{g})\frac{\Delta t}{2}$$

$$E(t + \Delta t) = E(t + \tfrac{1}{2}\Delta t) + (\mathbf{v} \cdot \rho\mathbf{g})\frac{\Delta t}{2}$$

At the end of this sequence, the conserved variables of the gas grid have been fully advanced to time $t + \Delta t$, accounting for both the internal fluid dynamics and the external force of gravity.

Spatial Reconstruction and the MUSCL Scheme

In a basic finite-volume grid, the simplest way to determine the state of the fluid at the interface between two cells is to assume the fluid properties (density, velocity, pressure) are constant across the entire cell. This is known as **Piecewise Constant** reconstruction.

While computationally cheap, piecewise constant reconstruction is highly diffusive. It acts like a blur filter, smearing out sharp shock waves across many grid cells. To capture the sharp, violent shocks of a cosmological simulation, we need a higher-order approach.

The **MUSCL (Monotonic Upstream-centered Scheme for Conservation Laws)** scheme upgrades our spatial resolution to second-order by replacing the flat constant states with **Piecewise Linear** slopes. Instead of assuming a fluid variable q (which represents $\rho, \mathbf{v}$, or P) is flat inside cell i, we calculate a gradient (slope) based on its neighbors, $i-1$ and $i+1$. We then use this slope, Δq_i, to linearly extrapolate the fluid's properties from the cell center to the left and right interfaces of the cell.

Mathematically, to find the fluid states on the exact left (L) and right (R) sides of the interface located between cell i and cell $i+1$, we extrapolate outwards from the cell centers:

$$q_{L,i+1/2} = q_i + \frac{1}{2}\Delta q_i$$

$$q_{R,i+1/2} = q_{i+1} - \frac{1}{2}\Delta q_{i+1}$$

However, a naive linear extrapolation creates a disastrous numerical artifact at shock fronts. When a steep slope tries to extrapolate across a discontinuous shock, it inevitably overshoots the true value, causing wild, unphysical oscillations (ringing) known as the Gibbs phenomenon.

To safely use linear reconstruction, we must introduce a **Slope Limiter**. A slope limiter acts as a localized safety switch. We first calculate two separate slopes for cell i: the backward difference (looking left) and the forward difference (looking right):

$$\Delta q_{\text{backward}} = q_i - q_{i-1}$$

$$\Delta q_{\text{forward}} = q_{i+1} - q_i$$

- If the fluid is smooth, these two slopes will be similar, and the limiter allows the second-order linear slope.
- If a shock is present, the slopes will violently disagree or change signs. The limiter detects this extremum and immediately forces the slope Δq_i to zero, temporarily dropping the local reconstruction back to a safe, flat first-order state just for that specific shock front.

A common and highly robust choice is the **Minmod limiter**. It compares the forward and backward slopes and selects the one with the smallest magnitude if they point in the same direction, returning zero if they point in opposite directions:

$$\text{minmod}(a,b) = \begin{cases} a & \text{if } |a| < |b| \text{ and } ab > 0 \\ b & \text{if } |b| < |a| \text{ and } ab > 0 \\ 0 & \text{if } ab \leq 0 \end{cases}$$

By setting our cell slope to $\Delta q_i = \text{minmod}(\Delta q_{\text{backward}}, \Delta q_{\text{forward}})$, we guarantee that the reconstructed values at the interfaces will never create new, artificial extrema. By applying this MUSCL reconstruction to the primitive variables $(\rho, \mathbf{v}, P)$, the simulation can resolve incredibly sharp shock fronts without sacrificing stability.

The HLLC Riemann Solver

Once the fluid states have been extrapolated to the left ($\mathbf{U}_L$) and right ($\mathbf{U}_R$) sides of an interface, the code must determine the flux of mass, momentum, and energy passing through it. This is done using an **approximate Riemann solver**.

When two different fluid states collide at an interface, they spawn a complex fan of waves propagating outwards. The Harten-Lax-van Leer (HLL) solver is a classic approximation that assumes this complex interaction can be simplified into just two waves: the fastest wave moving left and the fastest wave moving right. These wave speeds are denoted by the scalar variables S_L and S_R (where S stands for "Signal velocity", which should not be confused with the momentum density vector $\mathbf{S}$ used earlier in the macroscopic solver).

While incredibly stable, the standard HLL solver has a fatal flaw: it ignores the middle of the wave structure. Specifically, it completely misses the **Contact Discontinuity**—a wave where fluid density and temperature jump abruptly, but pressure and velocity remain continuous. Because HLL averages over this middle region, it heavily smears out contact discontinuities and shear flows, which are vital for resolving the intricate gas filaments of the cosmic web.

To fix this, modern cosmological codes use the **HLLC** solver (where "C" stands for Contact). The HLLC solver restores the missing middle physics by introducing a third wave speed, S_*, which represents the speed of the contact discontinuity. As the left and right waves move outward from the interface over the time step, they sweep out an expanding region of interacting gas. The S_* wave divides this middle region into two distinct "star" states: $\mathbf{U}_L^*$ and $\mathbf{U}_R^*$.

The calculation of the HLLC flux at the interface ($\mathbf{F}_{HLLC}$) proceeds as follows:

1. Estimate the Signal Velocities First, the local sound speeds for the left ($c_{s,L}$) and right ($c_{s,R}$) states are calculated using the adiabatic index γ, pressure P, and density ρ:

$$c_s = \sqrt{\frac{\gamma P}{\rho}}$$

These sound speeds, along with the normal velocities of the fluid ($v_{n,L}$ and $v_{n,R}$), are used to estimate the outermost wave speeds bounding the Riemann problem. To guarantee numerical stability, the solver must encompass the fastest possible waves traveling in either direction. A robust and standard estimate achieves this by taking the minimum and maximum possible signal velocities across the interface:

$$S_L = \min(v_{n,L} - c_{s,L}, v_{n,R} - c_{s,R})$$
$$S_R = \max(v_{n,L} + c_{s,L}, v_{n,R} + c_{s,R})$$

2. Calculate the Contact Wave Speed (S_*) The speed of the middle contact wave is derived directly from the conservation of momentum across the interface:

$$S_* = \frac{P_R - P_L + \rho_L v_{n,L}(S_L - v_{n,L}) - \rho_R v_{n,R}(S_R - v_{n,R})}{\rho_L(S_L - v_{n,L}) - \rho_R(S_R - v_{n,R})}$$

3. Compute the Star States Using the contact wave speed S_*, the solver can calculate the exact conserved variables for the intermediate "star" regions ($\mathbf{U}_L^*$ and $\mathbf{U}_R^*$) wedged between the shock fronts and the contact discontinuity.

For either the Left or Right state (denoted by the subscript K, where $K \in \{L, R\}$), the density of the star state is determined by mass conservation across the outermost wave:

$$\rho_K^* = \rho_K \left(\frac{S_K - v_{n,K}}{S_K - S_*} \right)$$

Next, we must determine the velocities and energy of these star states by looking at the physics of the contact discontinuity itself. This middle wave (S_*) is the literal boundary where the two original gas masses meet. Because they are pushing against each other with matching pressure and velocity, they do not mix; the interface acts as an impermeable wall that moves with the fluid.

Since no mass actually flows across this boundary, any property tied strictly to the fluid mass—specifically the transverse velocities (v_{t1}, v_{t2}) and the specific internal energy (ie/ρ)—remains constant and simply slides along with the flow. Meanwhile, the normal velocity of the fluid, by definition of moving perfectly with the boundary, becomes exactly S_*.

Therefore, the full vector of conserved variables for the star state $\mathbf{U}_K^* = [\rho^*, (\rho v_n)^*, (\rho v_t)^*, E^*, ie^*]_K$ is given by:

$$\mathbf{U}_K^* = \rho_K^* \begin{bmatrix} 1 \\ S_* \\ v_{t,K} \\ \frac{E_K}{\rho_K} + (S_* - v_{n,K}) \left(S_* + \frac{P_K}{\rho_K (S_K - v_{n,K})} \right) \\ \frac{ie_K}{\rho_K} \end{bmatrix}$$

4. Calculate the Final Flux The solver determines the flux at the interface based on the direction of these three wave speeds:

- If $S_L \geq 0$, the entire wave structure is moving right. The flux is simply the original Left flux: $\mathbf{F}_L$.
- If $S_L < 0 \leq S_*$, the interface is inside the left star region. The flux is: $\mathbf{F}_L^* = \mathbf{F}_L + S_L(\mathbf{U}_L^* - \mathbf{U}_L)$.
- If $S_* < 0 \leq S_R$, the interface is inside the right star region. The flux is: $\mathbf{F}_R^* = \mathbf{F}_R + S_R(\mathbf{U}_R^* - \mathbf{U}_R)$.
- If $S_R \leq 0$, the entire wave structure is moving left. The flux is the original Right flux: $\mathbf{F}_R$.

By properly resolving the contact wave, the HLLC solver accurately tracks the boundaries between hot and cold gas, allowing the simulation to capture the complex, multi-phase thermodynamics of galaxy formation.

The Dual Energy Formalism

In cosmology, gas falling into a dark matter halo routinely hits hypersonic speeds, reaching Mach 10 to Mach 100+ (because the primordial intergalactic gas is very cold, its local speed of sound is exceptionally low, allowing infalling gas to achieve massive Mach numbers). At these extreme velocities, the macroscopic kinetic energy (E_{kin}) of the gas completely dominates its internal thermal energy (E_{int}).

This creates a catastrophic numerical problem. In a standard finite-volume solver, we only track the total energy ($E_{tot} = E_{kin} + E_{int}$). To find the gas pressure, we must extract the internal energy by subtracting the kinetic energy:

$$E_{int} = E_{tot} - E_{kin}$$

When E_{kin} is 99.999% of E_{tot}, the limits of floating-point arithmetic cause catastrophic cancellation. The subtraction destroys the precision of the thermal energy, often resulting in exactly zero or even negative internal energies, which instantly crashes the simulation with unphysical negative pressures.

To solve this, modern cosmological codes employ the **Dual Energy Formalism**. Alongside the standard conserved variables, the simulation tracks the internal energy density as an independent, actively advected fluid field. This independent field is updated by its own fluxes and the physical work done by compression or expansion ($-P\nabla \cdot \mathbf{v}$).

The code then employs a dynamic "switch" during the calculation of the primitive variables:

- If the local flow is subsonic or undergoing a strong shock (e.g., where thermal energy is $> 0.1\%$ of the total energy), the code relies on the standard Total Energy equation. This guarantees perfect energy conservation across shock fronts.
- If the local flow is hypersonic ($E_{int}/E_{tot} < 0.001$), the total energy calculation is deemed mathematically polluted. The code "switches" to calculating pressure strictly from the independently tracked internal energy.

This dual-tracking approach guarantees that the gas temperature remains physically valid even when plummeting into the deepest gravitational wells of the cosmic web.

Coupling Hydrodynamics to the Expanding Universe

In a static box, the hydrodynamic equations are only sourced by gravity and their own internal pressure. However, in a cosmological simulation, the gas, just like the dark matter, is defined in **comoving coordinates** and is therefore subject to the physics of the expanding universe.

To correctly model this, the standard Euler equations must be modified with new "source terms" that account for the expansion. These terms are mathematically identical to the ones used for the N-body particles.

The cosmological effects are applied as "source terms" in the main integrator "Kick" steps, alongside the gravity. During each "Kick" step, an **external source acceleration** is applied to the gas in every grid cell, which is the sum of two components:

1. **Modified Gravity:** The comoving gravitational acceleration, $\mathbf{g}_{\text{comoving}}$, is calculated from the total density of *both* dark matter and gas. This acceleration is then scaled by $1/a^3$ to account for the dilution of physical density as the universe's volume ($V \propto a^3$) increases.
2. **Hubble Drag:** A velocity-dependent "friction" term, $-2H\mathbf{v}$, is added. This term, where H is the Hubble parameter and $\mathbf{v}$ is the gas's peculiar velocity, correctly damps the gas's peculiar momentum as space stretches.

The **external source acceleration** applied to the gas in each cell is therefore:

$$\mathbf{a}_{\text{source}} = \frac{\mathbf{g}_{\text{comoving}}}{a^3} - 2H\mathbf{v}$$

This acceleration, which is a force per unit mass, is then used to update the gas grid's conserved quantities over the half-time-step ($\Delta t/2$):

- **Momentum Update:** The momentum density $\mathbf{S} = \rho\mathbf{v}$ is updated by the force density ($\rho\mathbf{a}_{\text{source}}$):

$$\Delta\mathbf{S} = (\rho\mathbf{a}_{\text{source}})\frac{\Delta t}{2}$$

- **Energy Update:** The total energy density E is updated by the power density (Force · Velocity) delivered by these forces:

$$\Delta E = (\mathbf{v} \cdot \rho \mathbf{a}_{\text{source}}) \frac{\Delta t}{2}$$

By applying these cosmological source terms to the gas grid within the same KDK integrator as the dark matter particles, we ensure that both components feel the same gravity and the same cosmic expansion, allowing them to evolve in a physically consistent manner.

Initial Conditions for the Gaseous Component

In cosmological simulations, the baryonic gas and the dark matter must be initialized to reflect the physical reality of the universe long after the epoch of recombination. By the time a typical simulation begins, the gas has already spent millions of years falling into the gravitational potential wells excavated by the dark matter.

Therefore, the gaseous component must be initialized in lockstep with the dark matter, sharing its exact density fluctuations and bulk velocity flows, governed by the Zel'dovich approximation.

1. Initial Density The gas density, ρ_{gas}, is not uniform; it must perfectly trace the initial density perturbations of the dark matter. Once the dark matter particles are displaced to their starting positions, their mass is mapped to the Eulerian grid (via schemes like Cloud-in-Cell) to establish the primordial dark matter density field. The gas density in each cell is then set directly proportional to this field, scaled by the cosmic baryon fraction—the ratio of total gas mass to total dark matter mass in the simulation.

2. Initial Peculiar Velocity The gas cannot start "at rest". Because it has been gravitationally influenced by the dark matter for millions of years prior to the start of the simulation, the gas shares the same large-scale velocity flows. To achieve this synchronization, the continuous primordial velocity field is calculated natively on the high-resolution Eulerian grid using the Zel'dovich approximation. The gas grid is directly populated with these continuous velocity vectors, $\mathbf{v}_{\text{pec}}$. The collisionless dark matter particles, rather than carrying the grid, simply sample their individual initial velocities from this same continuous background field based on their starting coordinates.

3. Initial Energy Because we employ the **Dual Energy Formalism**, we must initialize two separate energy fields. First, the explicitly tracked *internal* energy is initialized to a very low, uniform baseline. This ensures the gas starts "cold," meaning its thermal pressure is negligible and too weak to artificially resist the initial gravitational collapse. Second, the *total* energy of the gas grid is initialized as the sum of this internal thermal energy and the macroscopic kinetic energy ($E_k = \frac{1}{2}\rho v^2$) dictated by its initial primordial momentum.

This setup creates a physically robust initial state. At $t = 0$, the simulation consists of two distinct but perfectly synchronized fluids: a collisionless dark matter component and a hydrodynamic gas component, both sharing the exact same primordial density peaks and velocity flows. When the simulation begins, the cosmic web collapses cohesively, with the gas naturally shocking and heating as it flows into the deepening dark matter halos.

Validation of the Hydrodynamic Solver

While the N-body component of our simulation is validated by checking its adherence to conservation laws (energy, momentum) in a static universe, validating the hydrodynamic component is more complex. The goal is not simply to conserve energy; in fact, hydrodynamic shocks are *designed* to convert kinetic energy into thermal energy, a process that must be captured correctly.

A hydrodynamic solver is instead validated by its ability to accurately reproduce the known analytical solutions to a set of classic, standardized test problems. These tests are the "unit tests" of computational fluid dynamics.

Conservation in a Closed Box

The most fundamental test is to ensure the solver correctly conserves all quantities in the absence of external forces.

- **The Setup:** A periodic, non-expanding box is initialized with a random distribution of gas densities, pressures, and velocities. The gravitational solver is turned off.
- **The Validation:** The simulation is run for many time steps. At each step, the code must verify that the following total quantities, summed over all grid cells, remain constant to machine precision:
  1. **Total Mass:** $M_{total} = \sum_i \rho_i L^3$
  2. **Total Momentum:** $\mathbf{P}_{total} = \sum_i (\rho \mathbf{v})_i L^3$
  3. **Total Energy:** $E_{total} = \sum_i E_i L^3$

- **What it Proves:** This test confirms that the numerical flux calculations are perfectly balanced—that any mass, momentum, or energy that leaves one cell correctly enters its neighbor, with no numerical "leaks" or "sources."

The 1D Shock-Tube

This test has a known, exact analytical solution (the **Sod Shock Tube** is the most famous variant) that validates the code's ability to handle all three fundamental wave structures.

- **The Setup:** A 1D tube of gas is initialized with a "diaphragm" at its center. The gas on the "Left" state has a high density and pressure, while the gas on the "Right" state has a low density and pressure. At $t = 0$, the diaphragm is removed.
- **The Expected Result:** The collision of the two states generates a self-similar wave structure (meaning its shape remains perfectly constant as it stretches over time). This structure is complex, splitting into three distinct features (see below).
- **The Validation:** After evolving the system to a time t, a snapshot of the simulation's density, pressure, and velocity along the 1D line is plotted. This numerical result must be compared directly against the known, exact mathematical solution. A successful test will correctly capture the speed, position, and amplitude of the three key features:
  1. A **Shock Wave** (an abrupt, discontinuous compression) propagating into the low-density region.
  2. A **Rarefaction Fan** (a smooth, continuous expansion) propagating back into the high-density region.
  3. A **Contact Discontinuity** (a jump in density, but not pressure) separating the two materials. Capturing this specific feature sharply, without excessive smearing, is the primary benchmark for the **HLLC Riemann solver**.

The Sedov-Taylor Blast Wave (Point Explosion)

This is the classic multi-dimensional test for how a code handles a powerful, symmetric explosion, such as a supernova.

- **The Setup:** A uniform, low-density gas fills the grid, initially at rest. At $t = 0$, a very large amount of thermal energy is deposited into a single central cell.
- **The Expected Result:** A strong, spherical shock wave propagates outwards from the center, sweeping the surrounding gas into a dense, hot shell. Because this explosion creates extreme hypersonic velocities (high Mach numbers), this is a premier stress-test for the **Dual Energy Formalism** and the **MUSCL slope limiters**, proving the code can handle violent energy conversions without crashing due to negative pressures.
- **The Validation:** This test has a known self-similar solution and is validated in two ways:
  1. **Symmetry:** The shock front must remain perfectly spherical. Any "boxy" or distorted shape indicates that the dimensional splitting in the solver is introducing errors.
  2. **Propagation Speed:** The radius of the shock front, R, must grow with time, t, according to a specific power law. For an explosion in a uniform medium, this is $R(t) \propto t^{2/5}$. A log-log plot of radius versus time must produce a straight line with a slope of $2/5$.

The Kelvin-Helmholtz Instability

This test is not about shocks, but about the code's ability to correctly model fluid mixing and the growth of instabilities.

- **The Setup:** A 2D box is initialized with two layers of fluid sliding past each other in opposite directions. For example, the top half has a velocity $v_x = +v$ and the bottom half has $v_x = -v$. A tiny, sinusoidal perturbation is introduced at the interface.
- **The Expected Result:** The shear at the interface is unstable. The small initial perturbation should grow exponentially, causing the interface to roll up into a characteristic series of vortices.
- **The Validation:** This is a test of the solver's numerical diffusion. Resolving these vortices correctly is a major victory for **second-order spatial reconstruction** and the **HLLC solver**. Simpler, first-order solvers (like standard HLL) introduce so much artificial viscosity that they smear out the interface, artificially damping the instability and preventing the vortices from ever forming.

Passing this standard suite of tests provides strong confidence that a hydrodynamic code is correctly solving the Euler equations and is ready for use in complex physical simulations.

Gas Physics

While the laws of hydrodynamics describe how gas moves, the true engine of galaxy formation is **thermodynamics**—the physics of how gas heats up and, more importantly, how it cools down. The balance between these two processes acts as a cosmic thermostat, determining whether a gas cloud has enough pressure to resist gravity or whether it will collapse to form stars.

Temperature

First, it must be understood what "temperature" means in the near-vacuum of interstellar or intergalactic space. In the air around us, temperature is a measure of the energy transferred by countless atoms constantly colliding with each other. In the extremely sparse gas of the cosmos, particles are so far apart that they rarely ever collide.

In this context, **temperature** is a direct measure of the **average kinetic energy** of the gas particles relative to the local bulk velocity of the fluid. It is a statement about **how fast the particles are moving**, not how much they are interacting.

- A **"hot"** gas is one where the individual atoms and ions are moving at very high random speeds. The gas inside a galaxy cluster, for example, can reach millions of degrees, even though it is less dense than any vacuum we can create on Earth.
- A **"cold"** gas is one where the particles are moving relatively slowly.

Gravitational Compression and Shocks

Cosmic gas doesn't have a "stove" to heat it up. Its temperature increases when energy is added to it from large-scale astrophysical processes. The primary heating mechanism is **gravitational compression**.

As gas is pulled into the deep gravitational well of a dark matter halo, it accelerates to enormous speeds. When this rapidly falling gas meets the gas that has already accumulated, it collides violently, creating an immense **shock wave**. This shock wave is an almost instantaneous conversion of the gas's ordered, in-falling kinetic energy into disordered, random motion—in other words, heat. This process, known as **virial heating**, can raise the gas temperature to millions of degrees, creating the vast, hot atmospheres we observe in galaxy clusters.

Other significant heating sources include:

- **Supernova Feedback:** The explosive death of massive stars creates powerful blast waves that rip through the surrounding medium, shocking and heating the gas.
- **Radiation:** High-energy photons from stars and active galactic nuclei can ionize atoms, transferring their energy to the gas and heating it.

Radiative Cooling

A gas cloud in the vacuum of space cannot cool down by touching anything cold. The *only* way it can lose energy is by radiating it away in the form of **photons** (light). This process is called **radiative cooling**, and it is the single most important mechanism for galaxy formation. If a gas cloud cannot cool, its internal pressure will forever resist the pull of gravity, and stars will never form.

Gas particles turn their kinetic energy into light through two main processes:

1. **Line Emission:** In a warm gas, collisions (even if rare) can knock an electron in an atom or ion into a higher energy level. When the electron inevitably drops back down, it emits a photon of a very specific wavelength or "line." This photon escapes into space, carrying away a small packet of the cloud's energy.
2. **Bremsstrahlung ("Braking Radiation"):** In very hot, ionized gas (a plasma), a fast-moving free electron can fly past a positive ion. The ion's electric field will deflect the electron, causing it to "brake" and change direction. The kinetic energy lost by the electron during this braking process is emitted as a high-energy photon (often an X-ray). This is the dominant cooling mechanism in the hot atmospheres of galaxy clusters.

The rate of cooling is highly dependent on the density and temperature of the gas. By implementing these heating and cooling functions in our simulation, we create a dynamic "cosmic thermostat" that, in a constant battle with gravity, ultimately dictates where and when the stars will begin to shine.

Key Literature & Further Reading
Teyssier, R. (2002). *Cosmological hydrodynamics with adaptive mesh refinement. A new high-resolution code called RAMSES.* arXiv:astro-ph/0111367. Available at https://arxiv.org/abs/astro-ph/0111367 **Riemann**

Solvers & MUSCL Schemes:
Toro, E. F. (2009). *Riemann Solvers and Numerical Methods for Fluid Dynamics: A Practical Introduction* (3rd ed.). Springer.

Adaptive Timestep

Computational cosmology simulations are filled with different components. Dark matter particles interact only through gravity, a long-range force that can be relatively slow. Baryonic gas, however, also interacts through hydrodynamic pressure, leading to shock waves and sound waves that propagate at very high speeds.

This creates a challenge: the simulation evolves on many different timescales simultaneously. In a dense, hot region of a gas cloud, the time it takes for a sound wave to cross a single grid cell might be a microsecond. In the cold, empty void, the time it takes for a particle to move significantly under gravity might be a million years.

If we were to use a single, "fixed" timestep (Δt) for the entire simulation, we would be forced to choose the *smallest* possible timescale—the microsecond from that one hot cell. This would grind the entire simulation to a halt, spending billions of calculations moving distant particles by imperceptible amounts.

The solution is the **adaptive timestep**. At every cycle, the shortest timescale required to maintain stability for every physical component is computed. The final timestep used to advance the simulation is the minimum of all of these, ensuring both physical accuracy and computational efficiency.

The Courant-Friedrichs-Lewy (CFL) Condition for hydrodynamics

For the grid-based hydrodynamic solver, the most restrictive limit is the **Courant-Friedrichs-Lewy (CFL) condition**. This condition is based on the principle that information (i.e., a sound wave or the fluid itself) must not be allowed to travel more than one grid cell (Δx) in a single timestep (Δt).

If a parcel of gas moves two cells in one step, the numerical solver for the adjacent cell never "sees" it, leading to a breakdown of the solution.

To enforce this, we must first find the maximum "signal velocity" (v_{signal}) anywhere in the simulation. This is the sum of the bulk fluid velocity (v) and the local sound speed (c_s). The sound speed itself depends on the gas pressure (P) and density (ρ) through the adiabatic index (γ):

$$c_s = \sqrt{\frac{\gamma P}{\rho}}$$

$$v_{\text{signal}} = v + c_s$$

The simulation must find the maximum signal velocity across all N cells, $v_{\max} = \max(v_{\text{signal},i} \text{ for } i \in [1, N])$. The CFL timestep limit is then the time it would take this fastest signal to cross one cell, scaled by a "safety factor" (C_{CFL}, typically 0.1 to 0.5) to ensure stability:

$$\Delta t_{\text{CFL}} = C_{\text{CFL}} \cdot \frac{\Delta x}{v_{\max}}$$

The Gravitational Timestep

For the N-body (particle) integrator, a different stability criterion applies. Particles in a strong gravitational field (e.g., near a dense halo or during a close encounter) experience high acceleration. If the timestep is too large, the integrator will "overshoot" the correct trajectory, artificially adding energy to the system and making it unstable.

The principle here is that the timestep must be a small fraction of the local dynamical time, which can be defined as the time it takes a particle to cross the gravitational softening length (ϵ) given its current acceleration.

This is a kinematic constraint. From basic kinematics ($x = \frac{1}{2}at^2$), the time to travel a distance ϵ under acceleration a is $t \sim \sqrt{\epsilon/|a|}$.

To ensure stability for all particles, the simulation must find the maximum acceleration experienced by any particle, $a_{\max} = \max(|a_i|)$. The gravitational timestep limit is then:

$$\Delta t_{\rm grav} = C_{\rm grav} \cdot \sqrt{\frac{\epsilon}{a_{\rm max}}}$$

Where $C_{\rm grav}$ is a dimensionless safety factor (e.g., 0.1 to 0.3) that controls the accuracy of the particle integration.

The Global Timestep

At each cycle, the simulation computes all relevant timestep limits. The **global timestep**, Δt, which will be used to advance *all* components (particles and gas) from time t to $t + \Delta t$, must be the single most restrictive (smallest) value.

Furthermore, it is common practice to introduce a user-defined maximum timestep, $\Delta t_{\rm max}$. This acts as a "ceiling," preventing the timestep from becoming excessively large in quiet regions of the simulation (which could reduce accuracy) and ensuring that data snapshots are saved at reasonably regular intervals.

The final global timestep is therefore the minimum of all constraints:

$$\Delta t = \min(\Delta t_{\rm CFL}, \Delta t_{\rm grav}, \Delta t_{\rm max})$$

This ensures that the simulation proceeds as fast as possible while respecting the most stringent physical constraints present anywhere in the computational domain, guaranteeing that no part of the simulation—from the fastest shock wave to the slowest drifting particle—evolves into an unstable, non-physical state.

Key Literature & Further Reading

Bryan, G. L., Norman, M. L., O'Shea, B. W., et al. (2014). *ENZO: An Adaptive Mesh Refinement Code for Astrophysics*. The Astrophysical Journal Supplement Series, 211(2), 19. arXiv:1307.2265. Available at https://arxiv.org/abs/1307.2265